UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

PEDRO F. LEÃO

LUCAS P. FLORES

DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE BAIXO CUSTO UTILIZANDO HLS EM FPGA

CURITIBA

2026

PEDRO F. LEÃO
LUCAS P. FLORES

DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE BAIXO CUSTO UTILIZANDO HLS EM FPGA

Development of a Low-Cost High-Frequency Trading System using HLS on FPGA

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Carlos Raimundo Erig Lima

CURITIBA
2026

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

PEDRO F. LEÃO
LUCAS P. FLORES

DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE BAIXO CUSTO UTILIZANDO HLS EM FPGA

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Data de aprovação: 13/abril/2026

Prof. Carlos Raimundo Erig Lima
Orientador
Universidade Tecnológica Federal do Paraná

Membro da banca a definir
A definir
Universidade Tecnológica Federal do Paraná

Membro da banca a definir
A definir
Universidade Tecnológica Federal do Paraná

Membro da banca a definir
A definir
Universidade Tecnológica Federal do Paraná

CURITIBA

2026

Aos familiares, professores e colegas que
sustentaram a trajetória acadêmica que tornou
este trabalho possível.

AGRADECIMENTOS

Agradecemos ao Prof. Carlos Raimundo Erig Lima pela orientação, pelas discussões técnicas e pela confiança no desenvolvimento de um tema que combina arquitetura de computadores, sistemas embarcados e aplicações financeiras de baixa latência.

Agradecemos à Universidade Tecnológica Federal do Paraná, Campus Curitiba, pela formação acadêmica e pelo ambiente de aprendizagem que permitiu integrar conhecimentos de programação, eletrônica digital, redes de computadores, síntese de hardware e metodologia científica.

Agradecemos também aos colegas, familiares e amigos que contribuíram com apoio, revisão, testes, questionamentos e incentivo durante a elaboração deste trabalho.

Em sistemas de baixa latência, a previsibilidade não é apenas uma propriedade de desempenho: é parte da própria corretude operacional.

RESUMO

Este trabalho apresenta o desenvolvimento de uma plataforma acadêmica de baixo custo para criação, geração e validação de estratégias de *High-Frequency Trading* (HFT) com uso de *High-Level Synthesis* (HLS) por meio do MATLAB HDL Coder. O objetivo central é permitir que a infraestrutura de comunicação, livro de ofertas, integração ARM–FPGA e validação permaneça previamente preparada, de modo que o desenvolvedor concentre sua intervenção na escrita de uma função de estratégia em MATLAB compatível com síntese. A plataforma utiliza a Intel/Terasic DE10-Nano, composta por processador ARM no HPS e lógica programável Cyclone V. No fluxo proposto, um gerador publica mensagens sintéticas codificadas em FAST por TCP; um receptor em C++ decodifica os dados, normaliza símbolos, lados, preços e quantidades e envia quadros de 256 bits por MMIO; no FPGA, um livro de ofertas simplificado calcula melhor compra, melhor venda, *spread* e desbalanceamento; por fim, a estratégia escrita em MATLAB é convertida para VHDL pelo HDL Coder, encapsulada por um wrapper RTL e integrada ao motor de decisão. A metodologia envolve definição do contrato ARM–FPGA, isolamento da estratégia como bloco substituível, geração automatizada de HDL, validação por testes C++, simulações VHDL com GHDL e preparação de métricas de latência, vazão, recursos FPGA e esforço de desenvolvimento. Os resultados demonstram a viabilidade funcional do fluxo como plataforma de co-projeto hardware/software e como ambiente para experimentar estratégias de trading geradas a partir de MATLAB, embora o sistema ainda não constitua uma plataforma de negociação real nem realize envio de ordens a bolsas.

Palavras-chave: fpga; *high-frequency trading*; síntese de alto nível; livro de ofertas; sistemas embarcados.

ABSTRACT

This work presents the development of a low-cost academic platform for creating, generating and validating High-Frequency Trading (HFT) strategies using High-Level Synthesis (HLS) through MATLAB HDL Coder. The main goal is to keep the communication infrastructure, order book, ARM–FPGA integration and validation flow previously prepared, so that the developer can focus on writing a synthesizable MATLAB strategy function. The platform targets the Intel/Terasic DE10-Nano, which combines an ARM processor in the HPS and Cyclone V programmable logic. In the proposed flow, a data-feed generator publishes synthetic FAST-encoded messages over TCP; a C++ receiver decodes the data, normalizes symbols, sides, prices and quantities, and sends 256-bit frames through MMIO; inside the FPGA, a simplified order book computes best bid, best ask, spread and imbalance; finally, the MATLAB strategy is converted to VHDL by HDL Coder, wrapped by RTL logic and integrated into the decision engine. The methodology covers the ARM–FPGA communication contract, isolation of the strategy as a replaceable block, automated HDL generation, C++ testing, VHDL simulation with GHDL and preparation of metrics for latency, throughput, FPGA resource usage and development effort. The results demonstrate the functional feasibility of the flow as a hardware/software co-design testbed and as an environment for experimenting with MATLAB-generated trading strategies, although the system is not yet a real trading platform and does not send orders to exchanges.

Keywords: fpga; high-frequency trading; high-level synthesis; order book; embedded systems.

LISTA DE FIGURAS

Figura 1 – Arquitetura conceitual do livro de ofertas simplificado 28

Figura 2 – Comunicação ARM–FPGA por filas circulares mapeadas em memória . 34

Figura 3 – Arquitetura geral da plataforma proposta 41

Figura 4 – Fluxo sugerido de criação de estratégia por MATLAB HDL Coder 45

Figura 5 – Fluxo de dados do protótipo 59

Figura 6 – Execução prevista do gerador e receptor 61

Figura 7 – Pipeline lógico de atualização do livro 66

LISTA DE FOTOGRAFIAS

LISTA DE GRÁFICOS

Gráfico 1 – Comparação proposta de latência por caminho de processamento . . 35

Gráfico 2 – Vazão e ocupação de filas sugeridas para avaliação futura 71

Gráfico 3 – Uso de recursos FPGA sugerido para relatório pós-síntese 71

LISTA DE TABELAS

Tabela 1 – Quadro de evento enviado do ARM para o FPGA 47

Tabela 2 – Quadro de resposta enviado do FPGA para o ARM 48

Tabela 3 – Mapa de registradores principal da ponte ARM–FPGA 60

Tabela 4 – Resumo da execução funcional na DE10-Nano 62

Tabela 5 – Registradores da interface MMIO 77

Tabela 6 – Formato do quadro ARM–FPGA 78

Tabela 7 – Variáveis de ambiente do receptor 82

LISTA DE QUADROS

Quadro 1 – Comparação entre eixos da literatura e o protótipo 39

Quadro 2 – Principais elementos da plataforma 42

Quadro 3 – Matriz de validação funcional do protótipo 55

Quadro 4 – Espaço reservado para métricas experimentais 56

Quadro 5 – Critérios de aceitação funcional 68

Quadro 6 – Riscos técnicos remanescentes e mitigação 70

LISTAGEM DE CÓDIGOS FONTE

Listagem 1 – Trecho da saída do receptor com respostas FPGA–ARM 63

Listagem 2 – Cálculo dinâmico da base RX no wrapper C++ 64

Listagem 3 – Empacotamento do quadro ARM–FPGA 64

Listagem 4 – Comandos de teste local 81

Listagem 5 – Geração do bloco de estratégia por HDL Coder 81

Listagem 6 – Fluxo previsto para DE10-Nano 82

Listagem 7 – Execução na placa 82

LISTA DE ABREVIATURAS E SIGLAS

long

long

Siglas

LISTA DE SÍMBOLOS

Letras Latinas

Letras Gregas

Sobrescritos

Subscritos

Notações

SUMÁRIO

1 INTRODUÇÃO 20

LinkbordercolorLinkcolor1.1 Considerações iniciais 20

LinkbordercolorLinkcolor1.2 Problema de pesquisa 21

LinkbordercolorLinkcolor1.3 Objetivos 22

1.3.1 Objetivo geral 22

1.3.2 Objetivos específicos 22

LinkbordercolorLinkcolor1.4 Justificativa 23

LinkbordercolorLinkcolor1.5 Estrutura do trabalho 23

2 REFERENCIAL TEÓRICO 25

LinkbordercolorLinkcolor2.1 Negociação de alta frequência 25

2.1.1 Caminho crítico em sistemas de HFT 25

2.1.2 Determinismo e variabilidade 26

LinkbordercolorLinkcolor2.2 Livro de ofertas e dados de mercado 26

2.2.1 Eventos por nível e eventos por ordem 27

2.2.2 Sinais derivados do topo do livro 27

LinkbordercolorLinkcolor2.3 FPGA, SoC e plataforma DE10-Nano 29

2.3.1 Comparação entre CPU e FPGA 29

2.3.2 Papel do HPS 30

LinkbordercolorLinkcolor2.4 Síntese de alto nível e MATLAB HDL Coder . . 30

2.4.1 Restrições de código sintetizável 31

2.4.2 Por que não gerar todo o sistema por HLS 31

LinkbordercolorLinkcolor2.5 FAST, mFAST e fluxo TCP 31

2.5.1 Template SimpleMD 32

2.5.2 Limites do feed sintético 32

LinkbordercolorLinkcolor2.6 MMIO, Avalon-MM e filas circulares 32

2.6.1 Ordenação de publicação 33

2.6.2 Avalon-MM como camada de integração 33

2.6.3 MMIO versus DMA 34

LinkbordercolorLinkcolor2.7 Latência, vazão e determinismo 34

3 TRABALHOS RELACIONADOS 36

LinkbordercolorLinkcolor3.1 Negociação algorítmica e baixa latência 36
LinkbordercolorLinkcolor3.2 Aceleração de HFT com FPGA 36
LinkbordercolorLinkcolor3.3 Livro de ofertas e modelos de atualização . . . 37
LinkbordercolorLinkcolor3.4 Síntese de alto nível e fronteiras de projeto . . 38
LinkbordercolorLinkcolor3.5 Comparação com o protótipo desenvolvido . . 38
LinkbordercolorLinkcolor3.6 Posicionamento do trabalho 38
4 PROPOSTA 40
LinkbordercolorLinkcolor4.1 Visão geral da arquitetura 40
LinkbordercolorLinkcolor4.2 Elementos da plataforma 40
4.2.1 Elementos de software 41
4.2.2 Elementos de hardware 43
4.2.3 Elementos de infraestrutura 43
LinkbordercolorLinkcolor4.3 Decisões arquiteturais 43
LinkbordercolorLinkcolor4.4 Fluxo de criação de estratégias com MATLAB HDL Coder 44
LinkbordercolorLinkcolor4.5 Modos de operação 46
LinkbordercolorLinkcolor4.6 Tratamento de falhas e observabilidade 46
LinkbordercolorLinkcolor4.7 Escopo positivo e escopo negativo 47
LinkbordercolorLinkcolor4.8 Contrato de eventos ARM–FPGA 47
LinkbordercolorLinkcolor4.9 Mapeamento de símbolos e geração de dados 48
LinkbordercolorLinkcolor4.10 Regra de estratégia 48
5 MATERIAIS E MÉTODOS 49
LinkbordercolorLinkcolor5.1 Materiais 49
5.1.1 Plataforma de hardware 49
5.1.2 Linguagens e bibliotecas 49
5.1.3 Ferramentas de construção e validação 50
5.1.4 Ambiente Quartus e MATLAB 50
5.1.5 Viabilidade e delimitações 50
LinkbordercolorLinkcolor5.2 Métodos 51
5.2.1 Definição da plataforma de desenvolvimento 51
5.2.2 Definição do contrato de comunicação 51

5.2.3 Construção do fluxo de dados de mercado 52
5.2.4 Implementação do livro de ofertas em hardware 52
5.2.5 Geração e encapsulamento da estratégia 53
5.2.6 Validação 54
5.2.7 Método proposto para métricas experimentais 54
5.2.8 Critérios de revisão técnica 56
6 RESULTADOS 58
LinkbordercolorLinkcolor6.1 Escopo do sistema 58
LinkbordercolorLinkcolor6.2 Modelagem do sistema 58
6.2.1 Modelo de componentes 58
6.2.2 Modelo de estados das filas 59
6.2.3 Modelo de dados em ponto fixo 60
6.2.4 Modelo de registradores 60
LinkbordercolorLinkcolor6.3 Apresentação do sistema 61
6.3.1 Execução funcional na DE10-Nano 62
LinkbordercolorLinkcolor6.4 Implementação do sistema 63
6.4.1 Wrapper MMIO em C++ 63
6.4.2 Receptor FAST e empacotamento de eventos 64
6.4.3 Ponte compartilhada em VHDL 65
6.4.4 Livro de ofertas 65
6.4.5 Estratégia MATLAB HDL Coder 65
6.4.6 Projeto Quartus e componente customizado 67
6.4.7 Fluxo de build e dependências 67
6.4.8 Critérios de aceitação funcional 67
LinkbordercolorLinkcolor6.5 Discussões 68
6.5.1 Interpretação dos testes 68
6.5.2 Riscos técnicos remanescentes 69
6.5.3 Espaço reservado para resultados quantitativos 69
7 CONCLUSÃO 72
REFERÊNCIAS 74
APÊNDICE A CONTRATO DE COMUNICAÇÃO ARM–FPGA 77
LinkbordercolorLinkcolorA.1 Registradores 77

LinkbordercolorLinkcolorA.2 Regras de filas 77
LinkbordercolorLinkcolorA.3 Formato de evento 77
LinkbordercolorLinkcolorA.4 Formato de resposta 78
LinkbordercolorLinkcolorA.5 Checklist de alteração do contrato 78
APÊNDICE B COMANDOS DE CONSTRUÇÃO, TESTE E EXECUÇÃO . 81
LinkbordercolorLinkcolorB.1 Validação local 81
LinkbordercolorLinkcolorB.2 Geração da estratégia 81
LinkbordercolorLinkcolorB.3 Geração HDL e build para DE10-Nano . . . 81
LinkbordercolorLinkcolorB.4 Variáveis de ambiente 82
LinkbordercolorLinkcolorB.5 Sequência recomendada de depuração . . 82

1 INTRODUÇÃO

Mercados eletrônicos modernos são sistemas computacionais distribuídos nos quais a formação de preços depende de mensagens de oferta, cancelamento, execução e atualização de livro de ofertas que percorrem redes de baixa latência. Nesse cenário, a capacidade de receber dados de mercado, atualizar estado interno e produzir uma decisão em intervalos previsíveis tornou-se um requisito técnico central para aplicações de negociação algorítmica e, em especial, para HFT. A literatura de microestrutura descreve que a redução de latência altera a dinâmica competitiva dos participantes e influencia liquidez, descoberta de preços e comportamento do livro de ofertas (HASBROUCK; SAAR, 2013; HENDERSHOTT; JONES; MENKVELD, 2011; BROGAARD; HENDERSHOTT; RIORDAN, 2014).

Do ponto de vista de engenharia de computação, a dificuldade não está apenas em executar uma regra de negociação, mas em construir uma cadeia de processamento com baixo tempo de resposta, controle explícito de filas, representação adequada dos dados e integração entre software e hardware. CPUs de uso geral oferecem flexibilidade e ecossistema maduro, porém dependem de execução sequencial, hierarquia de memória, interrupções, escalonamento de sistema operacional e variabilidade temporal. FPGAs, por outro lado, permitem transformar partes do fluxo em circuitos dedicados, explorando paralelismo espacial e latência determinística no nível de ciclos de clock (HARRIS; HARRIS, 2013; PATTERSON; HENNESSY, 2017).

Este trabalho investiga essa alternativa por meio de uma plataforma acadêmica de baixo custo baseada na Intel/Terasic DE10-Nano, que combina processador ARM no HPS com lógica programável Cyclone V. A contribuição principal está no uso de HLS com MATLAB HDL Coder para transformar uma estratégia de trading escrita em MATLAB em um bloco VHDL integrável a uma infraestrutura previamente preparada. Nessa abordagem, a comunicação ARM–FPGA, o fluxo de dados de mercado, o livro de ofertas simplificado, os testes e o projeto Quartus formam a base do ambiente; o ponto de variação passa a ser a estratégia, que pode ser redigida em MATLAB e regenerada para hardware.

1.1 Considerações iniciais

O termo HFT é empregado neste trabalho para representar sistemas automatizados que processam dados de mercado e emitem decisões em janelas temporais muito reduzidas. A implementação desenvolvida não se conecta a uma bolsa real, não executa ordens financeiras e não deve ser interpretada como sistema de produção. Ela é uma plataforma experimental para estudar os elementos computacionais que aparecem em uma cadeia de HFT: recepção de mensagens, normalização de eventos, estrutura de livro de ofertas, decisão algorítmica, medição de comportamento e integração com lógica programável.

A escolha por uma plataforma de baixo custo tem relevância acadêmica. Infraestruturas profissionais de HFT utilizam enlaces especializados, placas de rede de latência ultrabaixa, *ker-*

nel bypass, co-localização e FPGAs de alto desempenho. Esses recursos são financeiramente inacessíveis para a maioria dos laboratórios de graduação. A DE10-Nano, apesar de limitada em frequência, memória e recursos lógicos quando comparada a placas industriais, oferece um ambiente suficientemente rico para estudar os princípios de integração entre ARM, FPGA, barramentos, síntese de hardware e processamento determinístico (Terasic Technologies, 2024; Intel Corporation, 2025).

O projeto em estudo adota uma divisão pragmática: o software ARM recebe e decodifica mensagens FAST sintéticas; a FPGA mantém um estado resumido do livro de ofertas e calcula sinais de mercado; a estratégia, descrita em MATLAB, decide entre não agir, comprar ou vender e é gerada para VHDL por HDL Coder. Essa divisão evita a tentativa de sintetizar toda a pilha de rede e toda a semântica de mercado em hardware, o que ampliaria excessivamente o escopo de um TCC. Ao mesmo tempo, preserva o ponto central do trabalho: facilitar a criação de estratégias de trading em um ambiente no qual a infraestrutura de execução já está preparada.

1.2 Problema de pesquisa

O problema tratado neste TCC pode ser formulado da seguinte forma: como construir, em uma plataforma FPGA de baixo custo, uma infraestrutura que permita implementar estratégias de trading com menor esforço de descrição RTL, preservando determinismo, paralelismo em hardware e integração com um fluxo de dados de mercado? Essa pergunta combina duas dificuldades. A primeira é arquitetural: sistemas baseados apenas em CPU sofrem com processamento sequencial, overhead do sistema operacional, variabilidade de escalonamento e limitações do modelo von Neumann. A segunda é metodológica: escrever toda a lógica de decisão diretamente em VHDL aumenta a barreira de entrada para testar novas estratégias.

A proposta deste trabalho responde a esse problema com uma separação clara de responsabilidades. A infraestrutura de comunicação, livro de ofertas, filas e interface com a DE10-Nano é preparada uma vez. A estratégia fica em MATLAB, usando uma assinatura controlada e tipos compatíveis com síntese. Assim, o usuário pode se concentrar na regra de decisão e usar o MATLAB HDL Coder para gerar o bloco de hardware correspondente. O benefício esperado é reduzir o custo de experimentação de estratégias, sem abrir mão da execução em FPGA.

Do ponto de vista técnico, os requisitos derivados desse problema são: baixa variabilidade temporal no trecho em hardware; representação numérica fixa para preços e quantidades; processamento de livro de ofertas em estrutura de profundidade limitada; interface ARM–FPGA documentada; validação por simulação e execução em placa; e planejamento de métricas de latência, vazão, uso de recursos e produtividade na alteração de estratégias.

1.3 Objetivos

Esta seção apresenta o objetivo geral e os objetivos específicos do trabalho, considerando a proposta de uma plataforma orientada à geração de estratégias por HLS.

1.3.1 Objetivo geral

Desenvolver e documentar uma plataforma de HFT de baixo custo, baseada em ARM e FPGA, cujo objetivo principal é facilitar a criação de estratégias de trading em MATLAB e sua geração para hardware por meio do MATLAB HDL Coder, mantendo previamente preparados o fluxo de dados de mercado, o livro de ofertas simplificado, a comunicação ARM–FPGA e a infraestrutura de validação.

1.3.2 Objetivos específicos

Os objetivos específicos são:

- estruturar um ambiente no qual a estratégia de trading seja o principal bloco substituível pelo usuário;
- definir um modelo de estratégia em MATLAB compatível com HDL Coder, utilizando tipos sintetizáveis, entradas numéricas e saída de ação;
- automatizar o fluxo de geração VHDL da estratégia a partir do código MATLAB;
- definir o contrato de comunicação entre software ARM e lógica FPGA, incluindo registradores, ponteiros, semântica de filas e formato dos quadros;
- implementar e explicar o fluxo de geração e recepção de mensagens FAST sintéticas utilizado como alimentação de mercado;
- manter, no FPGA, um livro de ofertas por níveis de preço com profundidade fixa, ordenação por lado e cálculo de melhor compra, melhor venda, *spread* e desbalanceamento;
- integrar a estratégia gerada por MATLAB HDL Coder ao motor VHDL por meio de uma interface estável;
- validar funcionalmente os componentes por testes C++, simulações VHDL e teste de fumaça entre gerador e receptor;
- propor métricas para avaliar latência, vazão, uso de recursos FPGA e esforço necessário para alterar uma estratégia.

1.4 Justificativa

A relevância do trabalho decorre da interseção entre arquitetura de computadores, sistemas embarcados, redes e aplicações financeiras de baixa latência. Em HFT, pequenas variações de tempo de resposta podem modificar a ordem de chegada de mensagens e, consequentemente, a decisão disponível ao participante. A literatura mostra que atividades de negociação algorítmica e baixa latência afetam liquidez e descoberta de preços, ainda que os efeitos dependam do mercado, do período e da estratégia adotada (HENDERSHOTT; JONES; MENKVELD, 2011; BROGAARD; HENDERSHOTT; RIORDAN, 2014; HASBROUCK; SAAR, 2013). Portanto, estudar a infraestrutura computacional subjacente é relevante para compreender tanto ganhos de desempenho quanto riscos técnicos.

Do ponto de vista pedagógico, o projeto força a integração de conteúdos normalmente vistos de forma separada. O receptor C++ exige programação de sistemas, sockets TCP, manipulação binária e interface com bibliotecas externas. O bloco FPGA exige VHDL, filas circulares, protocolos de barramento, controle de *handshake* e testbenches. A etapa MATLAB HDL Coder introduz HLS e as restrições de escrever código algorítmico sintetizável. Essa etapa é central porque reduz a barreira para experimentar novas estratégias: em vez de reescrever o motor em VHDL, o desenvolvedor altera a função MATLAB, executa a geração HDL e valida o bloco resultante dentro da infraestrutura existente.

A escolha por FPGA é justificada pelo tipo de carga de trabalho. Atualizações de livro de ofertas, empacotamento de quadros, cálculo de *spread*, comparação de limiares e geração de ação são operações regulares, com largura fixa e baixa necessidade de memória dinâmica. Essas características são favoráveis à implementação em hardware dedicado. Em contrapartida, decodificação de protocolo, reconexão TCP, mensagens de erro e interação com sistema operacional permanecem no ARM, onde a flexibilidade de software é mais apropriada. Esse co-projeto evita uma solução puramente acadêmica desconectada das restrições práticas de depuração e integração.

Além da motivação técnica, há impacto educacional e econômico. Educacionalmente, o trabalho aproxima sistemas embarcados, arquitetura de computadores, mercado financeiro, HLS e validação experimental em um único projeto. Economicamente, a escolha por uma DE10-Nano e por ferramentas acadêmicas permite avaliar uma alternativa de baixo custo frente a soluções comerciais especializadas. Cientificamente, o projeto contribui com uma análise prática de como ferramentas modernas de HLS podem ser aplicadas a um domínio sensível a latência, mas sem exigir que cada nova estratégia seja escrita manualmente em RTL.

1.5 Estrutura do trabalho

O Capítulo 2 apresenta o referencial teórico utilizado para compreender HFT, livro de ofertas, FPGA, HLS, MATLAB HDL Coder, FAST, MMIO, Avalon-MM, filas circulares e métricas

de desempenho. O Capítulo 3 discute trabalhos relacionados em negociação algorítmica, aceleração em FPGA, HLS e processamento de livros de ofertas. O capítulo de proposta descreve a plataforma desenvolvida, suas fronteiras, o fluxo de geração de estratégias e seus contratos de dados. O Capítulo 5 detalha materiais, ferramentas, métodos de desenvolvimento e planejamento de métricas. O Capítulo 6 apresenta o sistema obtido, a modelagem, a implementação e os espaços reservados para resultados quantitativos. Por fim, o Capítulo 7 sintetiza as contribuições, limitações e perspectivas futuras.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os conceitos necessários para compreender o protótipo desenvolvido. A organização acompanha a cadeia real do sistema: mercado e livro de ofertas, processamento de mensagens, arquitetura ARM–FPGA, comunicação por MMIO, projeto de hardware em VHDL e HLS, e critérios de desempenho.

2.1 Negociação de alta frequência

HFT é uma forma de negociação algorítmica caracterizada por tomada de decisão automatizada, alto volume de mensagens, horizonte temporal curto e forte dependência de latência. Em vez de avaliar fundamentos econômicos em períodos longos, sistemas desse tipo exploram mudanças rápidas no livro de ofertas, desequilíbrios de fluxo, arbitragem entre instrumentos e oportunidades temporárias de liquidez. A literatura associa HFT à infraestrutura de baixa latência, à automação de estratégias e à necessidade de responder a eventos de mercado antes que a informação se torne obsoleta (ALDRIDGE, 2013; HASBROUCK; SAAR, 2013).

Em uma implementação real, o caminho crítico inclui recepção de pacotes, decodificação de protocolo, normalização de mensagens, atualização de estado de mercado, execução da estratégia, controle de risco, roteamento de ordens e confirmação. Este trabalho cobre apenas uma parte desse caminho: dados sintéticos de mercado, livro de ofertas simplificado, decisão elementar e retorno de resposta ao software. A delimitação é importante porque evita afirmar que o protótipo realiza HFT operacional; ele reproduz elementos computacionais relevantes para estudar latência e arquitetura.

As vantagens de uma abordagem em hardware são a previsibilidade temporal e a exploração de paralelismo. Comparações, somas, deslocamentos em arranjos pequenos e empacotamento de palavras podem ocorrer em circuitos dedicados, sem depender de escalonamento de CPU. As limitações são menor flexibilidade, maior custo de depuração, necessidade de definir larguras fixas e dificuldade para lidar com protocolos complexos. Por isso, o projeto utiliza hardware para o núcleo regular do problema e software para a borda flexível.

2.1.1 Caminho crítico em sistemas de HFT

Em sistemas de negociação de baixa latência, o caminho crítico é a sequência mínima de operações entre a chegada de uma informação de mercado e a emissão de uma resposta. Mesmo quando a estratégia financeira é simples, a infraestrutura precisa executar diversas etapas: receber bytes da rede, identificar fronteiras de mensagem, decodificar campos, atualizar estado, avaliar condições, aplicar filtros de segurança, serializar uma resposta e transmiti-la. Cada etapa adiciona latência e, principalmente, variabilidade. Essa variabilidade é relevante

porque dois eventos com latências médias semelhantes podem ter comportamentos muito diferentes em cauda, isto é, nos piores casos.

Em uma pilha tradicional baseada em CPU, o caminho crítico atravessa camadas de software geral: driver de rede, pilha TCP/IP ou UDP/IP, cópias de buffer, chamadas de sistema, escalonador, cache, predição de desvio e sincronização entre threads. Técnicas profissionais como *kernel bypass*, afinidade de CPU e *busy polling* reduzem parte desse custo, mas aumentam complexidade e dependência de hardware específico. A alternativa com FPGA desloca parte do caminho para uma lógica síncrona, na qual comparações e atualizações podem ocorrer em ciclos previsíveis.

O protótipo desenvolvido não elimina a pilha de rede, pois a recepção TCP e a decodificação FAST continuam no ARM. Entretanto, ele isola o trecho de atualização de livro e decisão em hardware. Essa separação é coerente com o estágio do projeto: antes de acelerar rede e protocolo, é necessário provar que o contrato de eventos, o transporte por MMIO e o núcleo de estado em FPGA funcionam corretamente. Assim, o caminho crítico estudado é menor que o de uma plataforma profissional, mas suficiente para investigar os fundamentos de co-projeto.

2.1.2 Determinismo e variabilidade

Determinismo, neste trabalho, não significa que o sistema sempre responderá no mesmo tempo absoluto em execução Linux completa. Significa que o trecho descrito em hardware possui transições controladas por clock e quantidade limitada de estados. Uma atualização de livro com profundidade fixa não depende de alocação dinâmica, coleta de lixo, paginação de memória ou contenção de locks. O custo fica associado à profundidade configurada e ao caminho combinacional/síncrono gerado.

Essa propriedade é especialmente importante em aplicações de livro de ofertas. A cada evento, o sistema deve preservar invariantes: melhor compra abaixo da melhor venda, lados corretamente ordenados, quantidade nula removendo nível e resposta correlacionada à sequência recebida. Se a infraestrutura é não determinística, torna-se difícil diferenciar erro funcional de atraso ocasional. Ao manter a lógica central em VHDL e validá-la por testbench, o projeto reduz esse espaço de ambiguidade.

2.2 Livro de ofertas e dados de mercado

O livro de ofertas limitado, ou *limit order book*, é a estrutura que organiza intenções de compra e venda por preço e quantidade. O lado comprador contém ofertas de compra, com melhor compra no maior preço disponível; o lado vendedor contém ofertas de venda, com melhor venda no menor preço disponível. A diferença entre melhor venda e melhor compra é o *spread*. A quantidade disponível nos melhores níveis e seu desbalanceamento podem ser usados como

sinais simples de pressão compradora ou vendedora (GOULD *et al.*, 2013; CONT; STOIKOV; TALREJA, 2010).

Um livro completo em bolsa pode conter ordens individuais, prioridades por tempo, cancelamentos parciais, execuções e múltiplos tipos de evento. O protótipo adota uma abstração por nível de preço: cada evento informa símbolo, lado, preço e quantidade agregada. O FPGA mantém, para cada símbolo, vetores de preço e quantidade com profundidade fixa. No lado comprador, os preços são ordenados de forma decrescente; no lado vendedor, de forma crescente. Atualizações existentes alteram quantidade; quantidades nulas ou eventos de remoção retiram o nível; novas entradas deslocam níveis inferiores até o limite da profundidade.

A principal vantagem dessa modelagem é a determinização do custo computacional. Com número fixo de símbolos e profundidade fixa, o circuito pode ser descrito com laços de limite conhecido e memória estática. A principal limitação é a perda de fidelidade em relação a um livro de ofertas real, pois não há prioridade por ordem individual, identificador real de ordem, execução parcial ou múltiplas bolsas. Ainda assim, a abstração é adequada para avaliar o fluxo de dados e a integração FPGA.

2.2.1 Eventos por nível e eventos por ordem

Há duas formas comuns de representar atualizações de livro. Na representação por ordem, cada mensagem descreve uma ordem individual, identificada por um código, com preço, quantidade, lado e estado. Cancelamentos e modificações referem-se a esse identificador. Essa abordagem é mais fiel a mercados reais, mas exige estruturas de busca por identificador, controle de prioridade temporal e tratamento de execuções parciais. Em hardware, isso tende a exigir memórias associativas, tabelas de dispersão ou estruturas indexadas, aumentando o custo de síntese.

Na representação por nível, usada neste protótipo, cada mensagem atualiza diretamente uma quantidade agregada em determinado preço e lado. A operação é mais simples: encontrar o preço, substituir a quantidade, inserir o nível ou removê-lo. O custo computacional passa a depender da profundidade do livro, não do número total de ordens. A consequência é que o sistema perde a capacidade de reconstruir a fila de prioridade interna de cada preço. Para a plataforma proposta, essa simplificação é vantajosa porque permite discutir ordenação, *spread* e desbalanceamento sem implementar uma bolsa completa.

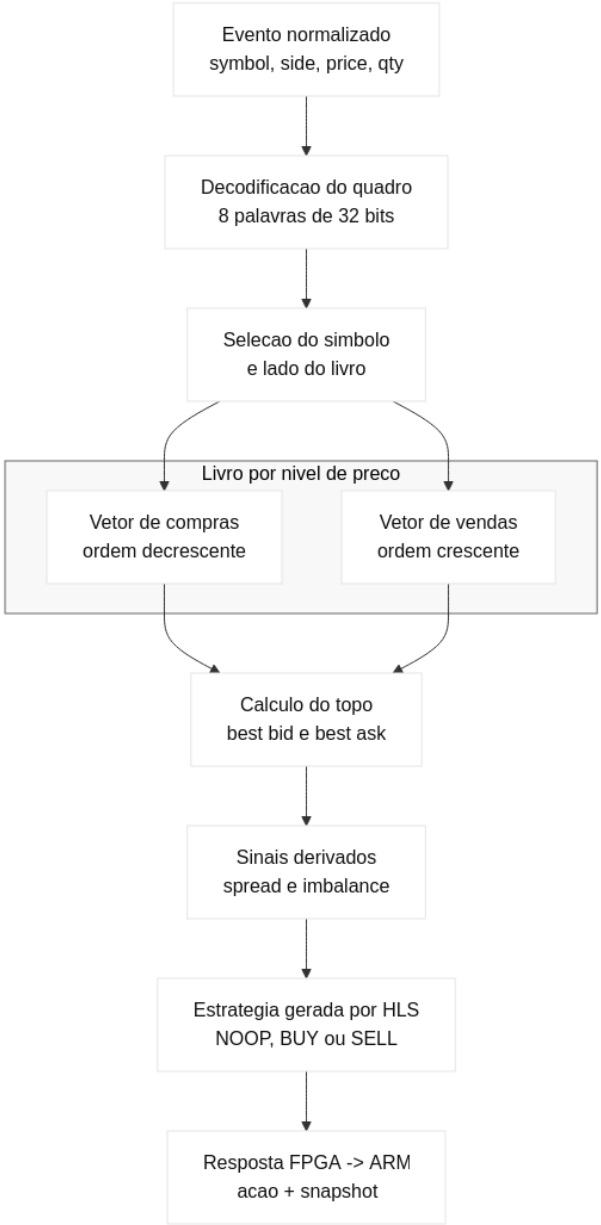
2.2.2 Sinais derivados do topo do livro

O topo do livro é formado pela melhor oferta de compra e pela melhor oferta de venda. A partir dele, o protótipo calcula o *spread* e o desbalanceamento de quantidade. O *spread* é calculado apenas quando ambos os lados existem e a melhor venda é maior que a melhor compra.

O desbalanceamento é calculado como a diferença assinada entre quantidade no melhor nível de compra e quantidade no melhor nível de venda. Valores positivos indicam maior quantidade compradora no topo; valores negativos indicam maior quantidade vendedora.

Esses sinais são simplificações de métricas usadas em microestrutura de mercado. Em sistemas reais, também poderiam ser considerados profundidade acumulada em múltiplos níveis, taxa de cancelamento, agressões recentes, tempo desde a última atualização, volatilidade local e posição atual da estratégia. O protótipo usa apenas sinais de topo porque eles cabem naturalmente em um quadro de resposta de oito palavras e são fáceis de validar por simulação.

Figura 1 – Arquitetura conceitual do livro de ofertas simplificado



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

2.3 FPGA, SoC e plataforma DE10-Nano

Uma FPGA é um circuito integrado reconfigurável formado por blocos lógicos, interconexões programáveis, registradores, memórias internas, multiplicadores e interfaces especializadas. Diferentemente de uma CPU, que executa instruções sequenciais sobre uma unidade funcional compartilhada, a FPGA implementa diretamente o caminho de dados e o controle da aplicação. Essa característica permite paralelismo no nível de operações e previsibilidade em ciclos de clock, desde que a aplicação seja descrita com recursos e temporização adequados (HARRIS; HARRIS, 2013).

A família Intel Cyclone V SoC combina lógica FPGA com um HPS baseado em ARM Cortex-A9, controladores de memória e pontes de comunicação entre processador e malha programável (Intel Corporation, 2025). A DE10-Nano utiliza esse SoC em uma placa educacional, com recursos suficientes para protótipos de sistemas embarcados e aceleração por hardware (Terasic Technologies, 2024). Para este trabalho, a característica mais relevante não é a quantidade absoluta de recursos lógicos, mas a existência de uma ponte HPS–FPGA acessível a partir do Linux embarcado.

O uso de um SoC FPGA traz um compromisso técnico. O ARM executa Linux, C++ e bibliotecas de protocolo com maior facilidade; a lógica programável executa operações regulares com latência previsível. A comunicação entre ambos, entretanto, torna-se parte crítica do projeto. Um acelerador muito rápido perde utilidade se o custo de transferência for alto, se o contrato de memória for ambíguo ou se houver divergência entre software, documentação e RTL.

2.3.1 Comparação entre CPU e FPGA

A CPU é otimizada para flexibilidade. Ela executa um fluxo de instruções, possui caches, unidades vetoriais, predição de desvios e suporte maduro a sistemas operacionais. Essa flexibilidade é ideal para protocolos complexos, tratamento de erro, alocação de objetos, logs e integração com bibliotecas. Contudo, a CPU compartilha unidades funcionais entre muitas tarefas e depende de uma microarquitetura projetada para desempenho médio elevado em cargas variadas.

A FPGA é otimizada para especialização. Um comparador, um somador e um registrador podem existir fisicamente como parte do circuito da aplicação. Se o projeto exige várias comparações independentes, elas podem ocorrer simultaneamente, desde que haja recursos lógicos suficientes. Essa especialização reduz flexibilidade: alterar a estrutura exige nova síntese e programação do bitstream. No protótipo, essa troca é aceitável porque o livro de ofertas tem profundidade pequena, formato fixo e operações bem definidas.

2.3.2 Papel do HPS

O HPS da DE10-Nano é o ponto de contato entre o mundo de software e a lógica reconfigurável. Ele executa Linux, acessa arquivos, abre sockets, usa bibliotecas C++ e mapeia regiões físicas. O FPGA, por sua vez, não conhece arquivos, processos ou sockets; ele observa sinais, registradores e barramentos. A fronteira entre os dois domínios é, portanto, uma tradução de abstrações: de mensagens e structs no software para palavras de 32 bits e handshakes no hardware.

Essa fronteira explica por que o projeto investe tanto no contrato MMIO. Se o software assume uma largura de quadro e o hardware sintetiza outra, o erro não aparece como exceção de alto nível; aparece como palavra trocada, ponteiro inconsistente ou resposta incorreta. A disciplina de documentação e teste é o mecanismo usado para manter o HPS e o FPGA sincronizados.

2.4 Síntese de alto nível e MATLAB HDL Coder

HLS é o processo de transformar uma descrição algorítmica de alto nível em uma implementação de hardware em RTL. Em vez de escrever manualmente todos os registradores, estados e sinais de controle, o projetista descreve operações em uma linguagem como C, C++ ou MATLAB, e a ferramenta gera VHDL ou Verilog. O MATLAB HDL Coder permite gerar código HDL a partir de funções MATLAB e modelos Simulink, desde que o código respeite restrições de sintetizabilidade, tipos fixos e tamanhos conhecidos (The MathWorks, Inc., 2026b).

No projeto analisado, a síntese de alto nível é usada de forma conservadora. A ponte MMIO, o livro de ofertas e o empacotamento de respostas são escritos manualmente em VHDL. A função MATLAB `strategy.m` contém apenas a regra decisória baseada em melhor compra, melhor venda, *spread* e desbalanceamento. O script `hdl_generator.m` invoca o HDL Coder em modo automatizado e gera um arquivo VHDL para o bloco `strategy`. Essa separação reduz o risco de depender de código gerado em toda a arquitetura e mantém as interfaces críticas sob controle explícito.

A vantagem dessa abordagem é permitir que regras algorítmicas sejam alteradas em MATLAB, ambiente familiar para modelagem matemática. A limitação é que nem todo MATLAB é sintetizável: alocação dinâmica, strings, estruturas variáveis, bibliotecas não suportadas e tipos de precisão flutuante podem inviabilizar a geração. O projeto evita esses problemas usando `uint32`, `int32`, limiares constantes e operações condicionais simples. O uso de ponto fixo escalado por 10^4 preserva casas decimais de preço sem recorrer a ponto flutuante em hardware (The MathWorks, Inc., 2026a).

2.4.1 Restrições de código sintetizável

Uma função MATLAB convencional pode usar vetores de tamanho variável, estruturas heterogêneas, strings, objetos, arquivos e operações dependentes do ambiente de execução. Uma função sintetizável precisa ser muito mais restrita. As entradas e saídas devem ter tipos e larguras conhecidas; laços precisam ter limites definidos; a aritmética deve ser representável em hardware; e o comportamento não pode depender de recursos que só existem no interpretador ou no sistema operacional.

No projeto, a função `strategy.m` recebe seis sinais já calculados: melhor compra, quantidade de compra, melhor venda, quantidade de venda, *spread* e desbalanceamento. Ela não percorre o livro, não lê arquivos, não usa strings e não aloca memória. Essa escolha não é apenas uma simplificação algorítmica; é uma escolha de engenharia para manter o bloco gerado pequeno e previsível. O restante do sistema fornece à função exatamente os campos que ela precisa, reduzindo o risco de geração HDL inesperada.

2.4.2 Por que não gerar todo o sistema por HLS

Embora HLS seja útil, gerar toda a ponte ARM–FPGA por MATLAB HDL Coder seria uma decisão arriscada. Barramentos, registradores, filas e handshakes dependem de contratos precisos com software e Platform Designer. Esses elementos são mais transparentes quando escritos manualmente em VHDL. Além disso, a depuração de um erro de ponte gerada automaticamente pode ser mais difícil do que a depuração de um bloco RTL pequeno e explícito.

Por isso, o projeto separa infraestrutura e política. A infraestrutura de comunicação e armazenamento é manual. A política de decisão, que tende a mudar mais durante experimentos, fica em MATLAB. Essa divisão facilita estudos futuros: novas estratégias podem ser geradas sem alterar o mapa de registradores nem a forma de atualização do livro.

2.5 FAST, mFAST e fluxo TCP

FAST, sigla de *FIX Adapted for STreaming*, é um padrão de codificação binária orientado a reduzir o tamanho de mensagens financeiras ao explorar templates, operadores de cópia, incremento e delta (FIX Trading Community, 2006). Em mercados reais, protocolos desse tipo reduzem banda e custo de transmissão de dados de mercado. Na plataforma proposta, FAST é usado como elemento de realismo controlado: o gerador `fast_data_feed.cpp` cria mensagens sintéticas com símbolo, lado, preço, quantidade e sequência; o receptor `fast_receiver.cpp` decodifica as mensagens usando a biblioteca mFAST (Object Computing, Inc., 2026).

A comunicação de rede do protótipo é TCP local na porta 9001. O servidor transmite cada mensagem precedida por um comprimento em ordem de rede, permitindo que o cliente leia exatamente a quantidade de bytes necessária antes de chamar o decodificador. Essa decisão simplifica o ambiente experimental, pois evita lidar com multicast, UDP, perda de pacotes e *kernel bypass*. A limitação é que TCP e localhost não reproduzem a pilha de dados de mercado de uma bolsa real. Para um TCC focado na integração ARM–FPGA e na geração de estratégias por HLS, a simplificação é aceitável porque mantém o feed determinável e testável.

2.5.1 Template SimpleMD

O arquivo `SimpleMD.xml` define o formato das mensagens usadas no protótipo. Ele contém uma sequência `MDEntries` com campos de símbolo, lado, preço, quantidade e número de sequência. O símbolo e o lado usam operador de cópia, o preço usa delta e a sequência usa incremento. Essa estrutura reproduz, em escala reduzida, a ideia de que mensagens sucessivas de mercado possuem redundância explorável pelo codificador.

A biblioteca mFAST gera tipos C++ a partir desse template. Assim, o receptor pode acessar campos por métodos tipados em vez de manipular manualmente offsets binários. A vantagem é reduzir erros de parsing no C++. A desvantagem é introduzir uma dependência externa e um passo de geração no build. O `Makefile` trata essa dependência clonando, compilando e instalando mFAST localmente.

2.5.2 Limites do feed sintético

O feed sintético escolhe símbolos e lados pseudoaleatoriamente, usa preços base e aplica deslocamentos pequenos. Isso permite gerar um fluxo contínuo sem depender de dados proprietários. Entretanto, não há bursts realistas, leilões, mensagens de abertura, pacotes fora de ordem, perda de rede, múltiplas sessões ou sincronização de relógio. Assim, os resultados obtidos com esse feed devem ser interpretados como validação funcional, não como simulação estatística completa de mercado.

2.6 MMIO, Avalon-MM e filas circulares

MMIO é uma técnica em que registradores de dispositivos são expostos no espaço de endereçamento do processador. Leituras e escritas em endereços específicos se transformam em transações de barramento. No Linux embarcado da DE10-Nano, o receptor pode mapear uma região física por `/dev/mem`, desde que a base da ponte HPS–FPGA esteja correta. O projeto usa a variável de ambiente `HFT_FPGA_MMIO_BASE` para habilitar o caminho FPGA e `HFT_FPGA_MMIO_SPAN` para definir o tamanho da janela.

No lado FPGA, o componente de integração utiliza Avalon-MM, protocolo de memória mapeada empregado pelo Platform Designer da Intel para conectar mestres e escravos em sistemas SoC (Intel Corporation, 2024a; Intel Corporation, 2024b). O arquivo `hft_trade_engine_avalon_mm.vhd` adapta sinais Avalon, como endereço, leitura, escrita, dados e `waitrequest`, ao barramento simples usado internamente pelo motor HFT. A presença de *timeout* evita que uma transação do HPS fique bloqueada indefinidamente durante reset ou falha interna.

O contrato de comunicação entre ARM e FPGA é baseado em duas filas circulares: TX para ARM–FPGA e RX para FPGA–ARM. Cada fila é de produtor único e consumidor único. No TX, o ARM escreve dados no *slot* e publica o avanço de `TX_HEAD`; o FPGA consome e avança `TX_TAIL`. No RX, o FPGA escreve resposta e avança `RX_HEAD`; o ARM lê e avança `RX_TAIL`. A fila usa a convenção em que `next(head) == tail` significa cheia, de modo que a capacidade útil é $D-1$. Essa escolha desperdiça um elemento, mas simplifica a distinção entre cheia e vazia.

2.6.1 Ordenação de publicação

A ordem de publicação é uma propriedade de corretude. No envio ARM–FPGA, o software deve escrever todas as palavras do slot antes de atualizar `TX_HEAD`. Se o ponteiro for publicado primeiro, o FPGA pode consumir um quadro parcialmente escrito. Na recepção FPGA–ARM, o ARM deve ler todas as palavras do slot antes de atualizar `RX_TAIL`. Se o ponteiro for avançado antes da leitura, o FPGA pode sobrescrever a resposta antes de o software copiá-la.

O wrapper C++ usa barreiras de sincronização após escrita de registradores e slots. No VHDL, os ponteiros são atualizados em borda de clock e as memórias são lidas por índice. A combinação é simples, mas exige que a documentação permaneça alinhada com a implementação. Essa é a razão para o contrato compartilhado ser tratado como superfície crítica do sistema.

2.6.2 Avalon-MM como camada de integração

Avalon-MM é usado para conectar o motor HFT ao HPS no Platform Designer. O wrapper `hft_trade_engine_avalon_mm.vhd` recebe sinais de chipselect, endereço, leitura, escrita, byteenable, dados de escrita e retorna dados de leitura e `waitrequest`. Internamente, ele transforma essas transações em uma interface mais simples com endereço em bytes e sinais `mm_rd_i` e `mm_wr_i`.

O sinal `waitrequest` é importante porque o mestre Avalon precisa saber quando a transação foi concluída. Se o escravo mantiver `waitrequest` ativo indefinidamente, o HPS pode travar ao acessar o registrador. O projeto inclui uma máquina de estados com *timeout*,

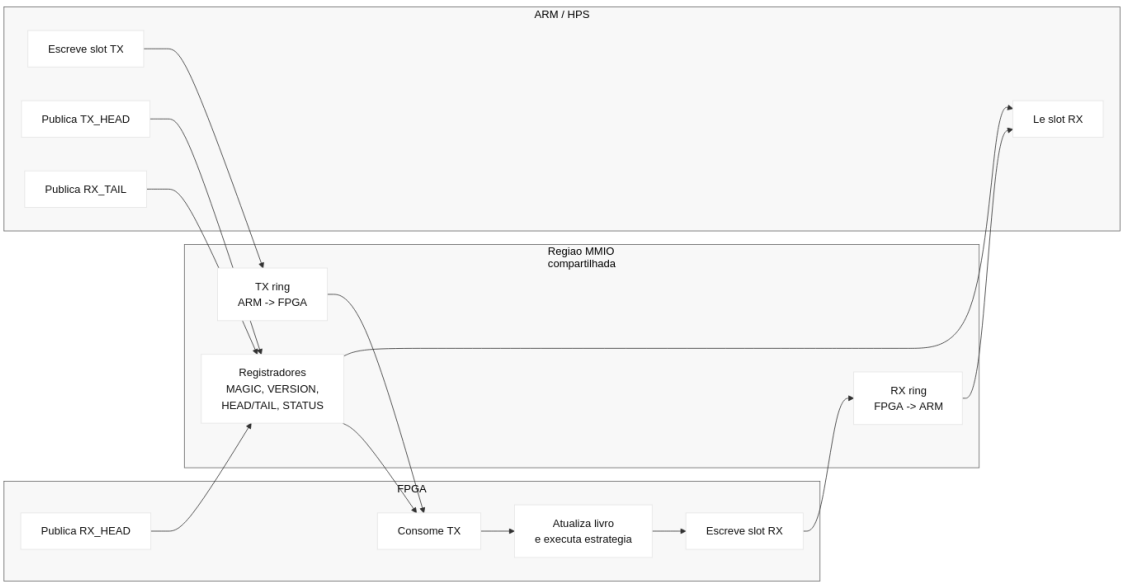
retornando sentinela em caso de ausência de resposta. Essa decisão aumenta robustez de depuração, especialmente durante reset ou integração inicial da placa.

2.6.3 MMIO versus DMA

O uso de MMIO com polling é apropriado para simplicidade e depuração. Cada palavra do quadro é escrita por acesso de memória mapeada, e os ponteiros são lidos periodicamente. Para fluxos muito intensos, essa abordagem pode consumir ciclos do ARM e limitar vazão. Uma alternativa seria DMA, no qual blocos maiores de dados são transferidos entre memória e FPGA com menor intervenção da CPU.

O projeto não usa DMA porque o objetivo inicial é validar o contrato funcional e a lógica de decisão. Introduzir DMA exigiria buffers coerentes, controle de descritores, interrupções ou polling de status, além de maior complexidade no software embarcado. Como trabalho futuro, DMA seria um passo natural para aproximar o protótipo de uma infraestrutura de maior vazão.

Figura 2 – Comunicação ARM–FPGA por filas circulares mapeadas em memória



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

2.7 Latência, vazão e determinismo

Latência é o intervalo entre a chegada de um evento e a disponibilidade da resposta correspondente. Vazão é a quantidade de eventos processados por unidade de tempo. Em sistemas de baixa latência, o valor médio não é suficiente; variância, piores casos e caudas de distribuição são igualmente importantes. Uma solução com latência média baixa, mas picos imprevisíveis, pode ser inadequada para HFT.

No protótipo, a latência total pode ser decomposta em recepção TCP, decodificação FAST, empacotamento C++, escrita MMIO, consumo da fila TX, atualização do livro em FPGA, execução da estratégia gerada por HLS, escrita na fila RX, leitura MMIO e impressão da resposta. A implementação atual valida funcionalmente essas etapas, mas ainda requer instrumentação completa com carimbos de tempo de hardware. Por isso, os gráficos de desempenho apresentados como placeholders neste TCC representam propostas de medição a serem preenchidas quando a execução em placa estiver estabilizada.

Gráfico 1 – Comparação proposta de latência por caminho de processamento

[GRÁFICO 1 – Sugestão: comparar latência software-only, latência com FPGA habilitado e latência por etapa: decodificação, MMIO, livro de ofertas e estratégia.]

Fonte: Autoria própria (2026).

3 TRABALHOS RELACIONADOS

Os trabalhos relacionados são agrupados em três eixos: efeitos e características de negociação de baixa latência, aceleração por FPGA em HFT e modelagem de livro de ofertas. Essa separação ajuda a posicionar o protótipo: ele não propõe uma nova teoria de mercado, mas uma plataforma de engenharia para reproduzir parte da infraestrutura computacional discutida nesses trabalhos.

3.1 Negociação algorítmica e baixa latência

Hendershott, Jones e Menkveld (2011) analisam a relação entre negociação algorítmica e liquidez, indicando que automação pode melhorar algumas métricas de mercado ao reduzir custos de seleção adversa e aumentar eficiência de cotação. Brogaard, Hendershott e Riordan (2014) discutem HFT e descoberta de preços, mostrando que participantes de alta frequência podem contribuir para incorporação de informação aos preços, embora o debate permaneça dependente de contexto e desenho de mercado. Hasbrouck e Saar (2013) tratam especificamente de negociação de baixa latência, destacando que a velocidade passa a ser um componente estratégico da interação entre participantes.

Esses estudos justificam a relevância de investigar infraestrutura computacional. Eles, porém, concentram-se em análise econômica e estatística, não em como construir uma cadeia ARM–FPGA de dados de mercado. O presente trabalho complementa esse eixo ao estudar a implementação de uma plataforma que permite observar os efeitos computacionais de filas, protocolos, atualização de estado e decisão em hardware.

3.2 Aceleração de HFT com FPGA

Leber, Geib e Litz (2011) apresentam aceleração de HFT com FPGAs, destacando a possibilidade de deslocar funções sensíveis a latência para hardware reconfigurável. O argumento central é próximo ao adotado neste TCC: operações de mercado com formato fixo e fluxo repetitivo podem se beneficiar de circuitos dedicados. Boutros *et al.* (2017) exploram HFT baseado em FPGA com HLS, demonstrando que ferramentas de síntese de alto nível podem ser usadas para construir blocos de negociação com menor esforço de desenvolvimento em comparação à escrita manual integral em RTL. Lockwood *et al.* (2012) discutem uma biblioteca de baixa latência em FPGA para HFT, reforçando que a latência não depende apenas do algoritmo, mas também da forma de conexão entre rede, memória, barramento e lógica.

A diferença principal em relação ao presente trabalho é o escopo de custo, ferramenta e maturidade. Os trabalhos citados se aproximam de bibliotecas e plataformas de maior desempenho, enquanto este TCC utiliza uma DE10-Nano e privilegia um fluxo acadêmico em que a

estratégia é escrita em MATLAB e gerada por HDL Coder. Em vez de implementar MAC Ethernet, UDP multicast ou *kernel bypass*, o projeto usa TCP sintético e concentra a aceleração no livro de ofertas e na estratégia. Essa escolha reduz fidelidade operacional, mas torna o projeto reproduzível em laboratório de graduação.

Do ponto de vista de arquitetura de computadores, a motivação para FPGA também pode ser interpretada pela diferença entre execução sequencial e execução espacial. Em uma CPU, uma sequência de atualizações do livro de ofertas compartilha unidades funcionais, hierarquia de cache e controle de fluxo. Mesmo quando há paralelismo interno, como execução fora de ordem, predição de desvios e vetorização, o programador observa uma abstração sequencial. Na FPGA, comparadores, registradores e somadores podem ser alocados como estruturas físicas distintas, permitindo que partes do processamento ocorram simultaneamente. Essa vantagem, contudo, só aparece quando o problema possui regularidade suficiente para justificar hardware dedicado (PATTERSON; HENNESSY, 2017; HARRIS; HARRIS, 2013).

O presente protótipo se encaixa nesse perfil porque o quadro de entrada tem largura fixa, a profundidade do livro é parametrizada e a estratégia usa limiares simples. A parte menos regular, isto é, decodificação de mensagens FAST, tratamento de TCP e manipulação de strings, permanece em software. Essa divisão evita transferir para a FPGA tarefas que exigiriam muito controle e pouco ganho. Assim, o trabalho se diferencia de abordagens que tentam implementar toda a pilha de rede no hardware, mas também se diferencia de estudos puramente software por materializar um contrato de barramento e um núcleo RTL verificável.

3.3 Livro de ofertas e modelos de atualização

Gould *et al.* (2013) revisam conceitos fundamentais de livros de ofertas, incluindo melhor compra, melhor venda, profundidade, eventos e impacto de chegada de ordens. Cont, Stoikov e Talreja (2010) propõem modelagem estocástica da dinâmica do livro, indicando que chegadas, cancelamentos e execuções podem ser tratados como processos que alteram estado discretamente. Embora o protótipo não implemente modelo estocástico completo nem prioridade temporal real, ele adota o princípio de estado discreto atualizado por eventos.

O livro de ofertas adotado na plataforma é propositalmente limitado: os eventos são de nível de preço, a quantidade substitui o valor agregado no nível, a profundidade é fixa e o número de símbolos é parametrizado. Essa escolha permite implementação em VHDL com arrays estáticos e laços limitados. Em comparação com estruturas de software mais gerais, há perda de expressividade; em comparação com uma estrutura de hardware totalmente customizada para um mercado específico, há ganho de clareza e facilidade de teste.

3.4 Síntese de alto nível e fronteiras de projeto

HLS é uma técnica recorrente em projetos de aceleração porque permite expressar parte do comportamento em linguagem de nível mais alto, reduzindo o esforço de escrita RTL. O MATLAB HDL Coder segue essa linha ao converter funções compatíveis com síntese em VHDL ou Verilog (The MathWorks, Inc., 2026b). Entretanto, a literatura e a prática de projeto indicam que HLS não elimina a necessidade de decisões arquiteturais: tipos de dados, latência, paralelismo, interface, memória e protocolo continuam determinando a qualidade do circuito gerado.

Na plataforma proposta, essa fronteira aparece de forma clara. A estratégia é escrita em MATLAB, pois consiste em comparações e condições sobre sinais já normalizados. Já a ponte MMIO, as filas circulares e o livro de ofertas foram mantidos em VHDL manual. Essa escolha é compatível com o papel de cada bloco: a estratégia é uma política que tende a mudar com experimentos; a ponte é infraestrutura crítica que exige controle explícito de registradores, ponteiros e ordem de publicação. O uso parcial de HLS, portanto, não representa uma limitação acidental, mas uma decisão de engenharia.

Em relação a Boutros *et al.* (2017), a contribuição deste TCC está menos na busca por uma implementação HFT completa e mais na construção de um fluxo acessível para graduação. A ferramenta escolhida é MATLAB HDL Coder, o que favorece a escrita de estratégias em uma linguagem já usada em modelagem numérica. Essa escolha é coerente com a proposta de permitir que a infraestrutura esteja pronta e que o usuário altere prioritariamente a função de decisão.

3.5 Comparação com o protótipo desenvolvido

O Quadro 1 sintetiza a relação entre os trabalhos revisados e o protótipo. O objetivo não é estabelecer superioridade, mas evidenciar o espaço ocupado por este TCC: uma plataforma acadêmica, aberta, de baixo custo e orientada à integração hardware/software e à geração de estratégias por HLS.

3.6 Posicionamento do trabalho

O protótipo desenvolvido posiciona-se como uma ponte entre trabalhos conceituais de microestrutura e sistemas profissionais de baixa latência. Sua contribuição acadêmica não é competir com plataformas comerciais, mas demonstrar, em código aberto e com ferramenta acessível, como organizar a fronteira entre ARM e FPGA em uma aplicação de mercado. O trabalho evidencia contratos que costumam ser pouco visíveis em descrições abstratas: largura de quadro, ordem de palavras, ponteiros de fila, base dinâmica da região RX, tratamento de livro cruzado e validação por testbench.

Quadro 1 – Comparação entre eixos da literatura e o protótipo

Eixo	Ênfase na literatura	Tratamento neste TCC
Microestrutura e HFT	Efeitos de baixa latência sobre liquidez, descoberta de preço e competição entre agentes.	Uso do contexto de HFT como motivação computacional, sem pretensão de avaliar lucratividade ou impacto econômico.
Aceleração por FPGA	Redução de latência em funções críticas, frequentemente com rede especializada, plataformas profissionais e HLS em C/C++.	Uso de DE10-Nano, MMIO, filas circulares, livro simplificado e estratégia MATLAB gerada por HDL Coder.
Livro de ofertas	Modelos por ordem, por nível, estocásticos ou estatísticos, com eventos de chegada, cancelamento e execução.	Livro por nível agregado, profundidade fixa, múltiplos símbolos parametrizados e proteção contra livro cruzado.
HLS	Aumento de produtividade na geração de RTL, condicionado a restrições de código sintetizável.	Aplicação central à criação de estratégias em MATLAB, mantendo transporte e estado de mercado em VHDL manual.
Validação	Medições de latência, frequência, vazão, área e comparação com alternativas.	Validação funcional por testes C++ e testbenches VHDL; campanha quantitativa indicada como etapa seguinte.

Fonte: Autoria própria (2026).

Assim, a principal originalidade local está na integração: feed FAST sintético, receptor C++ com mFAST, wrapper MMIO, ponte VHDL de anéis, livro de ofertas simplificado, estratégia gerada por MATLAB HDL Coder e componente Avalon-MM para DE10-Nano. Essa combinação é adequada para um TCC porque permite discutir arquitetura, implementação e validação sem depender de dados financeiros proprietários ou de acesso a uma infraestrutura de bolsa.

A lacuna coberta, portanto, é pedagógica e experimental. Muitos trabalhos discutem HFT em escala profissional ou analisam seus efeitos de mercado, mas poucos materiais de graduação mostram, em detalhe, como um evento de mercado vira palavras MMIO, como essas palavras atravessam um barramento HPS–FPGA, como uma fila circular evita ambiguidade entre cheio e vazio, e como uma estratégia gerada por HLS pode ser encaixada em uma arquitetura RTL maior. Esse nível de detalhe é o diferencial esperado de uma monografia de Engenharia de Computação.

4 PROPOSTA

Este capítulo apresenta a proposta da plataforma: uma infraestrutura de baixo custo para experimentar estratégias de trading escritas em MATLAB e geradas para FPGA por meio do MATLAB HDL Coder. A arquitetura não busca implementar um sistema completo de HFT em produção; ela prepara o caminho de dados, o livro de ofertas, a comunicação ARM–FPGA e a validação para que o bloco de estratégia possa ser substituído com menor esforço.

4.1 Visão geral da arquitetura

O fluxo principal inicia no programa `fast_data_feed`, que publica mensagens sintéticas de mercado por TCP. Cada mensagem contém uma sequência de uma entrada com símbolo, lado, preço, quantidade e número de sequência. O receptor `fast_receiver` conecta-se ao servidor, usa mFAST para decodificar os bytes FAST e imprime os dados. Quando a variável de ambiente `HFT_FPGA_MMIO_BASE` está definida, o receptor também ativa o caminho FPGA: converte o símbolo textual em identificador inteiro, converte preço decimal para ponto fixo escalado por 10^4, codifica lado e tipo de evento, e escreve um quadro de oito palavras de 32 bits na fila TX.

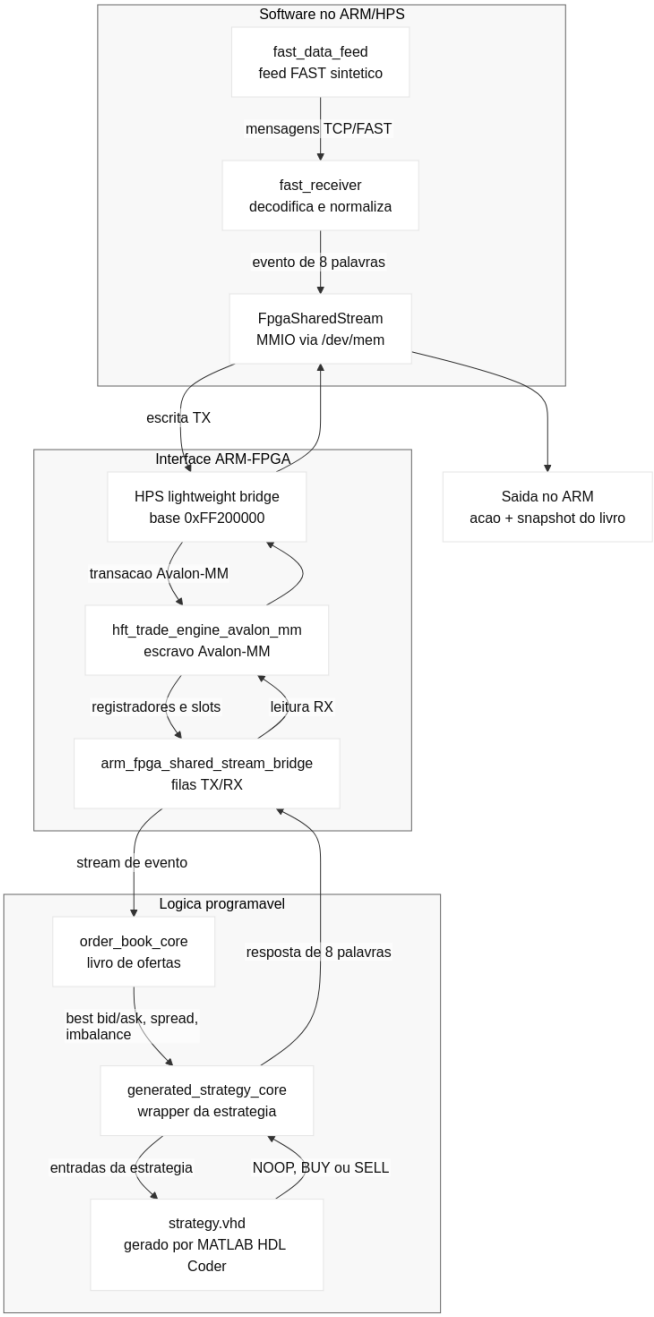
No FPGA, `arm_fpga_shared_stream_bridge.vhd` expõe registradores e duas regiões de memória para filas circulares. O bloco `hft_trade_engine.vhd` conecta essa ponte ao `trade_decision_core.vhd`. O núcleo de decisão, por sua vez, instancia `order_book_core.vhd` e `generated_strategy_core.vhd`. O livro de ofertas atualiza o estado por símbolo e produz um instantâneo com melhor compra, melhor venda, *spread* e desbalanceamento. A estratégia decide a ação e devolve um quadro de resposta para a fila RX. O software lê essa resposta e imprime os campos decodificados.

Essa arquitetura possui três fronteiras explícitas. A primeira fronteira é protocolo–software: bytes FAST são transformados em objetos C++ tipados. A segunda é software–hardware: objetos C++ são transformados em palavras de 32 bits e publicados por MMIO. A terceira é estado–decisão: eventos brutos são transformados em sinais derivados de livro, e esses sinais alimentam a estratégia. A separação em fronteiras facilita a validação porque cada etapa pode ser testada isoladamente.

4.2 Elementos da plataforma

O Quadro 2 resume os principais elementos da plataforma, sua função e sua integração com o sistema. Essa visão é importante para deixar claro quais partes formam a infraestrutura fixa e qual parte representa o ponto de variação do usuário: a estratégia escrita em MATLAB.

Figura 3 – Arquitetura geral da plataforma proposta



Fonte: Elaborado pelos autores a partir do diagrama Mermaid de arquitetura do projeto..

4.2.1 Elementos de software

O software possui dois executáveis principais. O primeiro, `fast_data_feed`, é um gerador de mercado. Ele não tenta simular todos os comportamentos de uma bolsa, mas fornece uma fonte contínua e controlada de mensagens. Sua vantagem é eliminar dependência de dados externos; sua limitação é não reproduzir bursts e assimetrias reais. O segundo, `fast_receiver`, é o ponto de integração. Ele decodifica, normaliza e envia eventos ao hard-

Quadro 2 – Principais elementos da plataforma

Elemento	Função no protótipo
`README.md`	Descreve o fluxo geral, comandos de construção, execução na DE10-Nano, arquitetura e formato dos quadros.
`Makefile`	Orquestra build, testes, geração HDL, cross-build ARM, Quartus e deploy.
`cpp/src/fast_data_feed.cpp`	Servidor TCP que gera mensagens FAST sintéticas de mercado.
`cpp/src/fast_receiver.cpp`	Cliente TCP que decodifica FAST, imprime eventos e, opcionalmente, envia quadros ao FPGA.
`cpp/src/fpga_shared_stream.h`	Wrapper C++ para abrir MMIO, validar cabeçalho, enviar e receber quadros por filas circulares.
`cpp/src/templates/SimpleMD.xml`	Template FAST usado para gerar tipos C++ da mensagem de mercado.
`vhdl/arm_fpga_shared_stream_bridge.vhd`	Ponte RTL com registradores e anéis TX/RX.
`vhdl/order_book_core.vhd`	Núcleo de livro de ofertas por nível de preço.
`vhdl/generated_strategy_core.vhd`	Wrapper que encapsula a estratégia gerada por MATLAB HDL Coder e adapta sua saída ao protocolo do motor.
`vhdl/hft_trade_engine.vhd`	Integra ponte, livro e estratégia em um motor único.
`vhdl/hft_trade_engine_avalon_mm.vhd`	Adapta o motor para um escravo Avalon-MM no Platform Designer.
`matlab/strategy.m`	Ponto principal de edição da estratégia de trading em MATLAB sintetizável.
`matlab/hdl_generator.m`	Script que automatiza a geração VHDL da estratégia via HDL Coder.
`quartus/de10_nano_hft`	Projeto Quartus/Platform Designer para integração na DE10-Nano.
`docs/arm-fpga-shared-memory-stream.md`	Documento de contrato entre ARM, FPGA, testes e interpretação de registradores.

Fonte: Autoria própria (2026).

ware. Também é responsável por manter o sistema operando mesmo quando o feed desconecta, pois implementa tentativa de reconexão.

O wrapper `FpgaSharedStream` é o componente de software mais sensível. Ele transforma uma região física em uma interface de alto nível com métodos `Send` e `Receive`. Isso reduz repetição no receptor e concentra regras de fila em um único arquivo. A classe também valida a geometria lida do FPGA, evitando que o software opere sobre um dispositivo incompatível. A existência de modo legado indica evolução do contrato; no texto final, o estado atual é o cabeçalho novo com `MAGIC` e `VERSION`.

4.2.2 Elementos de hardware

O hardware é organizado em camadas. A ponte compartilhada é a camada de transporte. O livro de ofertas é a camada de estado. A estratégia é a camada de decisão. O motor HFT integra essas camadas, e o wrapper Avalon-MM conecta o conjunto ao Platform Designer. Essa separação é uma boa prática porque permite testar cada bloco em isolamento antes de testar o fluxo completo.

O arquivo `order_book_core.vhd` é o núcleo de maior conteúdo algorítmico em VHDL. Ele contém procedimentos para limpar lado, remover nível e aplicar atualização ordenada. O arquivo `generated_strategy_core.vhd`, por outro lado, tem papel de adaptação: recebe snapshot, chama o bloco `strategy` gerado e monta a resposta. A separação entre livro manual e estratégia gerada é coerente com o objetivo de usar HLS sem abrir mão do controle do contrato de hardware.

4.2.3 Elementos de infraestrutura

O `Makefile` e os Dockerfiles são parte essencial do resultado, não apenas conveniência. Em projetos de FPGA e cross-compilation, diferenças de versão de compilador, biblioteca, simulador e ferramenta podem impedir reprodução. A plataforma define imagens para build C++/GHDL, cross-build ARM e MATLAB HDL Coder. Também define alvos específicos para simulação, geração de HDL, build Quartus, deploy e limpeza.

A pasta `quartus` contém o componente customizado e o projeto DE10-Nano. O arquivo `hft_trade_engine_avalon_mm_hw.tcl` declara parâmetros, arquivos HDL e interfaces do IP para o Platform Designer. Isso permite instanciar o motor HFT como escravo Avalon-MM. A vantagem é aproximar o protótipo de execução em placa; a limitação é a dependência de uma instalação Quartus e de uma configuração correta de HPS, clocks, reset e pontes.

4.3 Decisões arquiteturais

A primeira decisão arquitetural é manter a decodificação FAST no ARM. O protocolo FAST depende de templates, campos opcionais e representação binária compacta. Implementá-lo diretamente em FPGA exigiria uma máquina de estados maior, tratamento de variabilidade de mensagem e manutenção cuidadosa de compatibilidade com o template. Ao manter essa etapa em C++, o projeto preserva flexibilidade e usa a FPGA onde o formato já está normalizado.

A segunda decisão é usar anéis MMIO em vez de chamadas síncronas por registradores individuais. Uma alternativa simples seria escrever cada campo em um registrador e acionar um bit de início. Essa abordagem funciona para um único evento, mas acopla fortemente produtor

e consumidor: o ARM precisa esperar o FPGA terminar antes de escrever outro evento, ou o hardware precisa criar registradores de sombra. Com anéis, o software pode publicar vários eventos até a capacidade da fila, e o FPGA pode consumir quando estiver pronto. O custo é administrar ponteiros e capacidade útil `DEPTH - 1`.

A terceira decisão é separar livro de ofertas e estratégia. O livro é estado persistente; a estratégia é política de decisão. Misturar ambos em um único bloco dificultaria testar atualização de livro sem acionar regra de negociação, ou testar a regra com snapshots artificiais. A separação permite substituir a estratégia gerada por MATLAB sem reescrever o livro em VHDL. Também permite, futuramente, comparar múltiplas estratégias sobre o mesmo estado de mercado.

A quarta decisão é usar profundidades fixas. Em software, estruturas dinâmicas são naturais: vetores, mapas, árvores e filas podem crescer conforme necessidade. Em FPGA, crescimento dinâmico não existe da mesma forma; a área deve ser definida na síntese. O projeto usa `G_DEPTH`, `G_SLOT_WORDS`, `G_NUM_SYMBOLS` e `G_BOOK_DEPTH` para tornar essas escolhas explícitas. Isso facilita explorar configurações, mas cada alteração relevante exige recompilação e nova validação.

4.4 Fluxo de criação de estratégias com MATLAB HDL Coder

O objetivo principal da plataforma é reduzir o esforço necessário para implementar uma nova estratégia de trading em FPGA. Para isso, o ambiente separa a infraestrutura fixa da política de decisão. A infraestrutura fixa inclui recepção de mensagens, normalização, transporte MMIO, filas circulares, livro de ofertas simplificado, wrapper Avalon-MM e testbenches. A política de decisão é concentrada em uma função MATLAB, representada por `strategy.m`.

O fluxo esperado para o usuário é:

1. escrever ou alterar a função `strategy.m`, recebendo sinais numéricos do livro de ofertas, como melhor compra, melhor venda, quantidades, *spread* e desbalanceamento;

2. respeitar restrições de código sintetizável, como uso de tipos inteiros ou ponto fixo, ausência de alocação dinâmica, ausência de strings e controle de fluxo com limites conhecidos;

3. executar o script `hdl_generator.m` para gerar VHDL por meio do MATLAB HDL Coder;

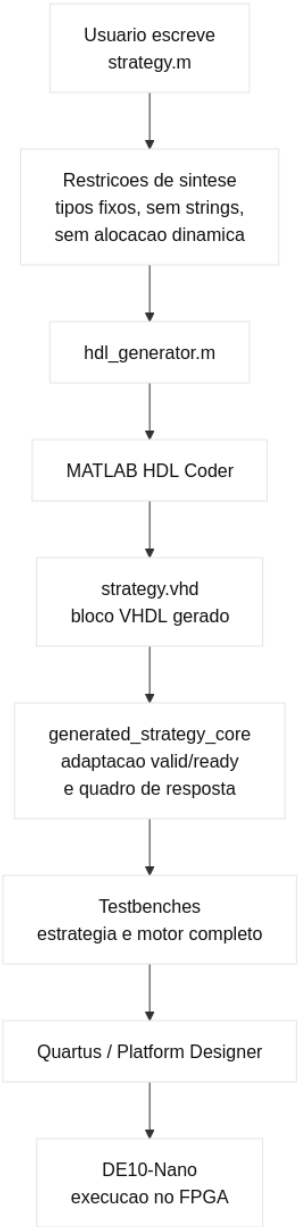
4. integrar o bloco gerado ao wrapper `generated_strategy_core.vhd`, que mantém a interface de resposta para o restante do motor;

5. executar os testbenches da estratégia e do motor completo para verificar se BUY, SELL e NOOP ocorrem nos cenários esperados;

6. somente após validação funcional, seguir para síntese Quartus e coleta de métricas.

Essa organização desloca o esforço do usuário para a definição algorítmica. Em vez de alterar filas, registradores, barramentos ou empacotamento de quadros, o usuário escreve uma regra em MATLAB dentro de uma assinatura conhecida. A vantagem é produtividade: estratégias simples podem ser descritas de forma mais próxima da notação algorítmica e depois convertidas para hardware. A limitação é que nem todo código MATLAB pode ser sintetizado. Estratégias com estruturas dinâmicas, bibliotecas não suportadas, aprendizado de máquina complexo ou uso intenso de ponto flutuante exigiriam adaptação para tipos fixos e arquitetura compatível com FPGA.

Figura 4 – Fluxo sugerido de criação de estratégia por MATLAB HDL Coder



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

4.5 Modos de operação

O protótipo possui dois modos práticos. No modo software-only, `fast_receiver` decodifica e imprime mensagens sem acessar FPGA. Esse modo é ativado quando `HFT_FPGA_MMIO_BASE` não está definido. Ele é útil para validar mFAST, socket TCP, template de mercado e comportamento do feed. Também serve como referência inicial para futuras comparações de latência de decodificação.

No modo FPGA, a variável `HFT_FPGA_MMIO_BASE` aponta para a base física do periférico. Opcionalmente, `HFT_FPGA_MMIO_SPAN` ajusta o tamanho mapeado e `HFT_FPGA_MMIO_DEV` altera o dispositivo usado, por padrão `/dev/mem`. Nesse modo, o receptor tenta abrir a ponte, validar `MAGIC`, ler geometria e enviar quadros. Se a ponte não estiver acessível, a falha ocorre no início, evitando que o programa opere sobre uma região incorreta.

Essa dualidade é metodologicamente útil. O modo software-only permite testar o lado de mercado em qualquer máquina de desenvolvimento. O modo FPGA exige placa, bitstream e permissões de acesso MMIO, mas valida a contribuição central do trabalho. A existência dos dois modos reduz o tempo de depuração, pois um erro de FAST não precisa ser confundido com erro de barramento.

4.6 Tratamento de falhas e observabilidade

O projeto contém mecanismos simples de observabilidade. O cabeçalho `MAGIC` confirma que a base mapeada corresponde ao periférico esperado. O campo `VERSION` permite detectar mudanças de protocolo. Os registradores de profundidade e largura permitem que o software recuse uma geometria incompatível. Os ponteiros de fila podem ser lidos para diagnosticar fila cheia, fila vazia ou consumidor travado.

No receptor, mensagens de erro distinguem falha de conexão, falha de decodificação, símbolo desconhecido, lado inválido e fila TX cheia. Essa distinção é importante porque cada problema tem causa provável diferente. Símbolo desconhecido aponta para divergência entre feed e tabela de normalização. Fila cheia aponta para FPGA lento, ponte indisponível ou receptor produzindo mais rápido que o consumo. Falha de `MAGIC` sugere endereço errado ou bitstream incompatível.

O wrapper Avalon-MM adiciona outra camada de robustez ao impedir que o barramento fique indefinidamente preso em certos cenários de teste. O uso de uma sentinela de timeout, representada por `BAADF00D`, ajuda a identificar transações que não completaram corretamente durante simulação. Em hardware real, uma política de timeout também é desejável para evitar que software embarcado fique bloqueado indefinidamente ao acessar um periférico mal inicializado.

4.7 Escopo positivo e escopo negativo

O escopo positivo inclui: geração sintética de dados de mercado, decodificação FAST, empacotamento em quadro fixo, transporte por MMIO, controle de fila circular, atualização de livro de ofertas por níveis, cálculo de sinais de topo de livro, decisão por limiares e preparação para síntese na DE10-Nano. Esses elementos estão materializados nos diretórios `cpp`, `vhdl`, `matlab`, `quartus` e `docs`.

O escopo negativo é igualmente relevante. O sistema não implementa uma conexão real com bolsa, não recebe dados multicast reais, não executa ordens, não possui camada de gerenciamento de risco, não implementa *kernel bypass*, não mede latência com relógio de hardware em produção, não gerencia múltiplos instrumentos com profundidade real de mercado e não garante conformidade regulatória. Essas ausências não são falhas do protótipo, mas delimitações necessárias para manter o projeto adequado a uma monografia de graduação.

4.8 Contrato de eventos ARM–FPGA

O quadro ARM–FPGA possui oito palavras de 32 bits, totalizando 256 bits. O formato vigente, observado em `fast_receiver.cpp` e documentado em `docs/arm-fpga-shared-memory-stream.md`, é:

Tabela 1 – Quadro de evento enviado do ARM para o FPGA

Palavra	Campo	Tipo	Descrição
0	`seq_no`	`uint32`	Número de sequência da mensagem sintética.
1	`symbol_id`	`uint32`	Identificador inteiro do símbolo, mapeado no ARM.
2	`price_1e4`	`uint32`	Preço multiplicado por 10^4.
3	`qty`	`uint32`	Quantidade agregada no nível.
4	`event_type`	`uint32`	1 para inserção/atualização, 2 para remoção, 3 para reinicialização.
5	`side`	`uint32`	1 para compra e 2 para venda.
6	`level_hint`	`uint32`	Reservado para indicação futura de nível ou ordem.
7	`reserved`	`uint32`	Reservado para extensão futura.

Fonte: Autoria própria (2026).

O quadro de resposta FPGA–ARM também possui oito palavras:

Esse contrato é o eixo de integração do trabalho. Qualquer mudança na largura do quadro, no significado das palavras ou na ordem de publicação deve ser refletida simultaneamente em C++, VHDL, testes e documentação.

Tabela 2 – Quadro de resposta enviado do FPGA para o ARM

Palavra	Campo	Tipo	Descrição
0	`seq_no`	`uint32`	Sequência ecoada para correlação.
1	`action`	`uint32`	0 para NOOP, 1 para BUY e 2 para SELL.
2	`best_bid_px`	`uint32`	Melhor preço de compra em escala 10^4.
3	`best_bid_qty`	`uint32`	Quantidade no melhor nível de compra.
4	`best_ask_px`	`uint32`	Melhor preço de venda em escala 10^4.
5	`best_ask_qty`	`uint32`	Quantidade no melhor nível de venda.
6	`spread_1e4`	`uint32`	Diferença entre melhor venda e melhor compra.
7	`imbalance`	`int32`	Diferença assinada entre quantidade de compra e venda.

Fonte: Autoria própria (2026).

4.9 Mapeamento de símbolos e geração de dados

O receptor mantém uma tabela fixa que traduz símbolos textuais para identificadores inteiros: AAPL, MSFT, NVDA, GOOGL e TSLA são mapeados para valores de 0 a 4. Essa decisão reduz a complexidade do FPGA, que não precisa comparar strings nem manipular codificação textual. O custo é a necessidade de manter a tabela sincronizada com o conjunto de símbolos do feed.

O gerador de dados cria preços próximos a um preço base por símbolo e escolhe lado, quantidade e deslocamento de preço pseudoaleatoriamente. Compras são geradas abaixo do preço base e vendas acima, reduzindo a probabilidade de livro cruzado. Mesmo assim, `order_book_core.vhd` possui proteção adicional: rejeita atualizações de compra que cruzariam a melhor venda e atualizações de venda que cruzariam a melhor compra.

4.10 Regra de estratégia

A estratégia atual é intencionalmente simples. Se um dos lados do livro está vazio, se o livro está cruzado ou se o *spread* supera 2,5000 unidades monetárias em escala 10^4, a ação é NOOP. Se o desbalanceamento é maior ou igual a 500, a ação é BUY. Se o desbalanceamento é menor ou igual a -500, a ação é SELL. Caso contrário, a ação permanece NOOP.

Essa regra não deve ser avaliada como estratégia financeira lucrativa. Sua finalidade é produzir uma decisão verificável e dependente do estado do livro de ofertas, permitindo testar se o pipeline de dados preserva campos, ordem de palavras, sinais e semântica. A vantagem é a simplicidade para validação; a limitação é a baixa representatividade de estratégias reais, que exigiriam controle de risco, previsão, custos, posição, *slippage* e conformidade.

5 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais e os métodos utilizados para construir e validar a plataforma proposta. A ênfase metodológica recai sobre o fluxo de criação de estratégias em MATLAB, geração VHDL por HDL Coder, integração com a infraestrutura ARM–FPGA e planejamento das métricas experimentais.

5.1 Materiais

5.1.1 Plataforma de hardware

A plataforma alvo é a DE10-Nano, baseada em Intel Cyclone V SoC. O uso dessa placa é justificado pela presença conjunta de processador ARM e FPGA, permitindo que Linux embarcado execute a pilha C++ enquanto a lógica programável execute o núcleo de atualização de livro e decisão. No projeto Quartus, o dispositivo configurado é `5CSEBA6U23I7`, e a entidade de topo do sistema Platform Designer é `hft`.

5.1.2 Linguagens e bibliotecas

O software foi implementado em C++11. Os arquivos centrais são `fast_data_feed.cpp`, `fast_receiver.cpp` e `fpga_shared_stream.h`. A biblioteca mFAST é usada para geração de tipos a partir do template `SimpleMD.xml` e para codificação/decodificação FAST. A escolha por C++ decorre da necessidade de manipular sockets, buffers binários, tipos inteiros e acesso a `/dev/mem` com baixo overhead.

O hardware foi descrito em VHDL, com testbenches também em VHDL. Os módulos principais são `arm_fpga_shared_stream_bridge.vhd`, `order_book_core.vhd`, `generated_strategy_core.vhd`, `trade_decision_core.vhd`, `hft_trade_engine.vhd` e `hft_trade_engine_avalon_mm.vhd`. O VHDL foi escolhido porque permite descrever explicitamente registradores, sinais, handshakes e memória interna.

O MATLAB foi utilizado para a regra de estratégia em `strategy.m` e para geração automatizada de VHDL por `hdl_generator.m`. O projeto não tenta gerar toda a arquitetura por HLS; o bloco de decisão é tratado como alvo principal de MATLAB HDL Coder. Essa decisão mantém a infraestrutura de barramento sob controle manual e concentra a flexibilidade no ponto que mais interessa ao usuário: a redação da estratégia.

5.1.3 Ferramentas de construção e validação

O `Makefile` é o ponto de entrada do fluxo. Ele orquestra construção de imagens Docker, instalação da mFAST, configuração CMake, testes C++, simulações VHDL, geração HDL via MATLAB, *cross-build* ARM, geração do Platform Designer e compilação Quartus. Docker é usado para reduzir variação de ambiente de desenvolvimento (Docker Inc., 2026). CMake organiza os alvos C++ e a geração de tipos FAST (Kitware, Inc., 2026). GHDL executa os testbenches VHDL (GHDL Project, 2026).

O projeto também contém `Dockerfile.cross-armhf` e `toolchains/arm-linux-gnueabihf-sysroot.cmake`, usados para compilar binários compatíveis com imagens Linux ARM antigas da DE10-Nano. O uso de sysroot evita vincular o binário final contra versões de `glibc` e `libstdc++` indisponíveis no alvo.

5.1.4 Ambiente Quartus e MATLAB

O ambiente Quartus é utilizado para transformar os módulos VHDL em um bitstream programável na FPGA. O fluxo de integração prevê geração do sistema Platform Designer por `qsys-generate`, seguida pelas etapas `quartus_map`, `quartus_fit` e `quartus_asm`. Essas etapas correspondem, respectivamente, a análise/síntese, posicionamento/roteamento e geração do arquivo de programação. O arquivo resultante esperado é `de10_nano_hft.sof`.

O ambiente MATLAB é encapsulado em Docker com a imagem `hft-matlab-hdl:r2025b`. O alvo `matlab-hdl-generate` executa `hdl_generator.m` e verifica se o arquivo VHDL gerado foi produzido. A geração é tratada como parte do build, não como ação manual de interface gráfica. Essa decisão é metodologicamente importante porque permite repetir o processo e reduz risco de divergência entre a estratégia MATLAB redigida pelo usuário e o VHDL efetivamente integrado ao sistema.

5.1.5 Viabilidade e delimitações

A viabilidade prática da plataforma decorre de quatro fatores. O primeiro é o uso da DE10-Nano, uma placa de baixo custo quando comparada a soluções FPGA comerciais para HFT. O segundo é a disponibilidade de ferramentas acadêmicas, especialmente MATLAB HDL Coder, Quartus e simuladores VHDL. O terceiro é o uso de dados simulados, o que elimina dependência de acesso a bolsas, contratos de dados proprietários e questões regulatórias de envio de ordens. O quarto é a modularização do sistema, que permite validar feed, comunicação, livro, estratégia e integração em etapas separadas.

Os principais riscos técnicos são timing constraints, limitação de recursos da Cyclone V, divergência entre o VHDL gerado e o wrapper de integração, e ausência inicial de métricas instrumentadas. Esses riscos foram tratados por meio de profundidades fixas, interfaces documentadas, testbenches e separação entre validação funcional e avaliação quantitativa. Quanto aos aspectos éticos, a plataforma usa apenas dados sintéticos e não realiza negociação real, de modo que não há interação com participantes de mercado nem execução de ordens financeiras.

5.2 Métodos

5.2.1 Definição da plataforma de desenvolvimento

A plataforma foi organizada em torno de uma separação entre infraestrutura e estratégia. A infraestrutura inclui comunicação, normalização de eventos, livro de ofertas, empacotamento de quadros, barramento Avalon-MM, testes e fluxo de build. A estratégia é mantida em MATLAB, com assinatura e tipos compatíveis com HDL Coder. Essa separação permite que um usuário altere a política de decisão sem modificar a ponte MMIO, o livro de ofertas ou o wrapper de integração.

Como a interface ARM–FPGA é binária, foram estabelecidos invariantes de integração: base RX dinâmica, largura de oito palavras por slot, tabela de símbolos consistente com o feed, publicação de ponteiros apenas após escrita do payload e profundidade compatível com a janela MMIO. Esses invariantes protegem o fluxo de geração de estratégias, pois uma estratégia nova só é útil se os dados de entrada chegarem ao bloco gerado com semântica estável.

5.2.2 Definição do contrato de comunicação

O contrato ARM–FPGA foi tratado como elemento central. Foram identificados registradores de cabeçalho (`MAGIC`, `VERSION`, `TX_DEPTH`, `RX_DEPTH`, `SLOT_WORDS`), ponteiros (`TX_HEAD`, `TX_TAIL`, `RX_HEAD`, `RX_TAIL`), bases de memória (`TX_BASE` e `RX_BASE`) e semântica das filas. A base RX é calculada dinamicamente como `TX_BASE + DEPTH * SLOT_WORDS * 4`, evitando o erro de hardcode observado em versões anteriores. A capacidade útil foi considerada como `DEPTH - 1`, devido à convenção de fila circular em que `next(head) == tail` representa fila cheia.

O contrato foi validado por leitura cruzada. No software, o método `Send` escreve o slot e depois atualiza `TX_HEAD`. No hardware, a ponte só consome quando há diferença entre cabeça e cauda e o consumidor está pronto. No caminho inverso, o FPGA escreve a resposta na RAM RX e avança `RX_HEAD`; o software lê o slot e avança `RX_TAIL`. Essa correspondência foi usada como critério de consistência entre texto, código e testbenches.

A definição do contrato também incluiu a identificação de responsabilidades de escrita. O ARM é o único produtor da fila TX e, por isso, é o único agente autorizado a publicar `TX_HEAD`. O FPGA é o consumidor da TX e publica `TX_TAIL`. Na fila RX, a responsabilidade se inverte: o FPGA produz respostas e publica `RX_HEAD`, enquanto o ARM consome respostas e publica `RX_TAIL`. Essa separação reduz ambiguidades porque cada ponteiro tem um único escritor. Os dois lados podem ler os quatro ponteiros, mas não competem pela escrita do mesmo registrador.

Outro aspecto metodológico foi a adoção de palavras de 32 bits como unidade de interoperabilidade. O ARM trabalha naturalmente com inteiros de 32 bits em C++, e o barramento Avalon-MM também expõe dados nessa largura no wrapper utilizado. Internamente, o VHDL concatena oito palavras em um vetor de 256 bits para transporte entre blocos. Essa conversão preserva simplicidade no mapa MMIO e, ao mesmo tempo, permite que o processamento de hardware trate o evento como um pacote único.

5.2.3 Construção do fluxo de dados de mercado

O gerador `fast_data_feed` foi estruturado como servidor TCP. A cada iteração, seleciona símbolo, lado, preço e quantidade, codifica uma mensagem FAST com mFAST e transmite o tamanho seguido do payload. O receptor `fast_receiver` implementa reconexão automática, leitura exata do quadro, decodificação FAST e impressão dos campos. No modo FPGA, o receptor converte esses campos para o quadro de oito palavras e chama `FpgaSharedStream::Send`.

Para evitar que a FPGA manipule texto, a etapa de normalização é feita antes do envio. Símbolos são convertidos para IDs e lado textual é convertido para código numérico. O preço decimal é convertido para inteiro escalado. Essa normalização é parte do método porque define a fronteira de responsabilidade: software lida com formatos ricos e variáveis; hardware lida com números fixos.

A escolha por TCP local não foi interpretada como tentativa de reproduzir rede de bolsa. Ela foi usada como mecanismo de teste para exercitar codificação, leitura de buffers e reconexão com menor complexidade operacional. O método evita introduzir dependências de multicast, placa de rede específica, permissões de captura ou dados proprietários. Como consequência, as conclusões sobre rede são limitadas: o trabalho avalia a estrutura de processamento e integração, não a latência de uma pilha de mercado real.

5.2.4 Implementação do livro de ofertas em hardware

O livro de ofertas foi implementado como arrays estáticos indexados por símbolo, lado e profundidade. A atualização procura preço existente, remove nível quando necessário, ou

insere novo preço na posição ordenada. A ordenação é decrescente para compras e crescente para vendas. Após cada evento aceito, o bloco calcula melhor compra, melhor venda, *spread* e desbalanceamento. O sinal `snapshot_valid_o` indica que um instantâneo está disponível para a estratégia.

Essa implementação privilegia determinismo e simplicidade. O custo de atualização é limitado por `G_BOOK_DEPTH`, e a profundidade padrão é 8. Para um sistema real, seria necessário avaliar frequência máxima, uso de LUTs, registradores e memória, além de técnicas de pipeline para aumentar vazão. No protótipo, a prioridade foi obter comportamento funcional verificável.

O método de atualização foi escolhido em vez de uma ordenação genérica. Uma ordenação completa a cada evento desperdiçaria recursos, pois a alteração ocorre em apenas um nível. A atualização incremental procura o preço afetado e desloca apenas os elementos necessários. Essa decisão reduz custo lógico e facilita explicar o pior caso, que fica limitado ao tamanho do vetor de níveis.

5.2.5 Geração e encapsulamento da estratégia

A estratégia foi escrita em MATLAB com tipos inteiros fixos e condições simples. O script `hdl_generator.m` configura o HDL Coder para VHDL, define argumentos de entrada e gera o bloco `strategy`. Em VHDL, `generated_strategy_core.vhd` encapsula esse bloco gerado e preserva a interface de *snapshot* e resposta. O wrapper também força NOOP quando o livro está vazio ou cruzado, adicionando uma proteção independente da função MATLAB.

O método de encapsulamento evita acoplar a assinatura do bloco gerado ao restante do sistema. Caso a estratégia MATLAB seja alterada, o wrapper continua sendo o ponto onde se traduzem sinais de livro para resposta de oito palavras. Essa camada é útil porque ferramentas HLS podem alterar detalhes de porta, latência ou nomes conforme configuração. Ao concentrar a adaptação em um único módulo, reduz-se o impacto de regenerar o núcleo de decisão.

O papel esperado do usuário é escrever a regra de decisão dentro desse limite. A função MATLAB pode combinar *spread*, desbalanceamento, preços e quantidades, desde que mantenha tipos e operações sintetizáveis. O fluxo proposto não exige que o usuário escreva VHDL para cada nova estratégia. O VHDL manual permanece como infraestrutura de execução; a função MATLAB representa a política que será testada e sintetizada.

Os limiares da estratégia foram tratados como constantes didáticas, não como parâmetros calibrados em dados reais. O valor de desbalanceamento igual a 500 e o limite de *spread* igual a 25000, em escala 10^4, servem para produzir ações observáveis nos testes. Em uma pesquisa quantitativa posterior, esses limiares deveriam ser definidos por análise de dados históricos, custo de transação, volatilidade do ativo e restrições de risco.

5.2.6 Validação

A validação foi dividida em níveis:

- teste C++ `fpga_shared_stream_test`, que usa arquivo temporário como região MMIO simulada e verifica envio, recepção, cálculo de base RX e fila cheia;
- teste de fumaça `cpp-smoke`, que executa feed e receptor em software;
- testbench básico da ponte compartilhada;
- testbench rápido da ponte, com rajadas, *wrap-around* e contrapressão;
- testbench do `order_book_core`, validando inserção, rejeição de livro cruzado e remoção;
- testbench do `generated_strategy_core`, validando ações e proteção contra livro cruzado;
- testbench do `hft_trade_engine`, que valida fluxo MMIO, anéis, livro e decisão;
- testbench do wrapper Avalon-MM, verificando comportamento de `waitrequest` e *timeout*.

Os comandos correspondentes são agregados por `make check` e `make vhdl-test-all`. Para execução em placa, o método prevê `make build`, programação do arquivo `.sof`, habilitação das pontes HPS–FPGA, `make deploy` e execução dos binários ARM com `HFT_FPGA_MMIO_BASE=0xFF200000`, ajustado pelo offset real do Platform Designer.

O Quadro 3 organiza os testes por risco mitigado. Essa matriz foi utilizada para verificar se cada elemento essencial da arquitetura possui pelo menos uma evidência de validação funcional.

5.2.7 Método proposto para métricas experimentais

Embora a implementação atual esteja mais forte em validação funcional do que em medição quantitativa, o método de avaliação foi definido para continuidade do trabalho. Para latência fim a fim, devem ser registrados quatro instantes: geração ou recebimento da mensagem no software, publicação do quadro TX, consumo do quadro pelo FPGA e disponibilidade da resposta RX. No ARM, esses instantes podem ser medidos inicialmente com relógio monotônico de alta resolução. No FPGA, a instrumentação recomendada é um contador de ciclos amostrado nos eventos de consumo e publicação.

Quadro 3 – Matriz de validação funcional do protótipo

Artefato	Risco técnico mitigado	Evidência de validação
`FpgaSharedStream`	Cálculo incorreto de bases, fila cheia mal detectada, avanço de ponteiro fora de ordem.	Teste C++ com backing file e profundidade reduzida.
`arm_fpga_shared_stream_bridge`	Divergência de mapa MMIO, perda de slot, erro de *wrap-around*.	Testbench básico e testbench rápido com rajadas e contrapressão.
`order_book_core`	Livro fora de ordem, remoção incorreta, aceitação de livro cruzado.	Testbench específico do livro de ofertas.
`generated_strategy_core`	Ação incompatível com snapshot, perda de resposta, livro inválido gerando ordem.	Testbench da estratégia e verificação de NOOP em casos inválidos.
`hft_trade_engine`	Falha de integração entre ponte, livro e estratégia.	Testbench de motor completo com escrita MMIO e leitura de resposta.
`hft_trade_engine_avalon_mm`	Interface Avalon travada, `waitrequest` indevido, transação sem retorno.	Testbench do wrapper Avalon-MM com sentinela de timeout.
`fast_data_feed/fast_receiver`	Incompatibilidade FAST, falha de decodificação ou empacotamento.	Teste de fumaça software-only e inspeção do empacotamento.

Fonte: Autoria própria (2026).

Para vazão, o método consiste em aumentar progressivamente a taxa do feed e observar três indicadores: retorno falso de `Send`, ocupação estimada da fila TX e atraso entre sequência de entrada e sequência de resposta. O ponto de saturação ocorre quando a chegada de eventos supera a capacidade de consumo do caminho hardware/software. A ocupação das filas pode ser calculada por diferença modular entre cabeça e cauda.

Para uso de recursos, o método depende dos relatórios do Quartus. Devem ser coletados número de elementos lógicos, registradores, blocos de memória, multiplicadores, frequência máxima e caminhos críticos. A comparação mais útil é incremental: ponte isolada, ponte mais livro, ponte mais livro e estratégia. Essa decomposição permite descobrir se o gargalo vem do barramento, da atualização de livro ou do bloco gerado por HLS.

Além de métricas clássicas de desempenho, este trabalho deve avaliar produtividade de criação de estratégias. Para isso, propõe-se registrar: número de linhas alteradas em `strategy.m`, tempo necessário para gerar o VHDL, quantidade de alterações manuais fora do MATLAB, número de falhas de síntese e tempo até uma estratégia passar no testbench. Essa métrica é coerente com o objetivo principal do TCC, pois mede se a plataforma realmente facilita a criação de estratégias por MATLAB.

Quadro 4 – Espaço reservado para métricas experimentais

Métrica	Como medir	Meta de referência	Resultado
Latência fim a fim	Timestamp no ARM entre recebimento do evento e leitura da resposta.	Comparar com versão software-only.	[PREENCHER]
Latência interna FPGA	Contador de ciclos entre consumo TX e publicação RX.	Jitter interno inferior a $1\ \mu$s.	[PREENCHER]
Vazão	Eventos por segundo até ocorrência de fila cheia ou perda de resposta.	Pelo menos 100.000 mensagens/s.	[PREENCHER]
Speedup	Comparação com implementação software-only equivalente.	Aceleração mínima de 5 vezes.	[PREENCHER]
Uso de recursos	Relatórios Quartus de LUTs, registradores, memória e frequência máxima.	Caber na Cyclone V da DE10-Nano.	[PREENCHER]
Produtividade HLS	Tempo e alterações necessárias para trocar a estratégia MATLAB.	Mudar a estratégia sem alterar barramento ou livro.	[PREENCHER]

Fonte: Autoria própria (2026).

As metas de referência foram definidas como critérios de sucesso para a campanha experimental, não como resultados já obtidos. Em particular, o speedup mínimo de 5 vezes depende de uma implementação software-only equivalente do livro e da estratégia, enquanto a vazão de 100.000 mensagens por segundo depende de ensaios com controle de taxa de chegada e coleta de ocupação das filas. Essa distinção evita confundir validação funcional com avaliação quantitativa.

5.2.8 Critérios de revisão técnica

A revisão técnica do trabalho utilizou quatro critérios. O primeiro foi rastreabilidade: toda afirmação sobre arquitetura deveria apontar para código, documento técnico ou comportamento de teste. O segundo foi não extrapolação: componentes inexistentes, como envio real de ordens ou controle de risco completo, não foram apresentados como implementados. O terceiro foi alinhamento de contrato: C++, VHDL, documentação e testes deveriam concordar sobre largura de quadro, ordem de palavras e base de registradores. O quarto foi adequação acadêmica: cada tecnologia citada deveria ter definição, uso no projeto, vantagem, limitação e integração.

Esses critérios são especialmente importantes porque o tema HFT pode induzir exageros. Um protótipo acadêmico de baixo custo não deve ser descrito como infraestrutura de

negociação profissional. A contribuição aqui é demonstrar uma cadeia funcional e testável para estudar aceleração e comunicação ARM–FPGA.

6 RESULTADOS

Este capítulo apresenta o sistema obtido, sua modelagem, aspectos de implementação e validação. Os resultados são principalmente funcionais, pois a campanha quantitativa completa de latência, vazão e uso de recursos ainda depende da instrumentação descrita no subseção 5.2.7.

6.1 Escopo do sistema

O sistema desenvolvido é uma plataforma de pipeline HFT assistida por FPGA, com foco na geração de estratégias por MATLAB HDL Coder. Ele recebe dados sintéticos de mercado, normaliza eventos em software, transporta quadros por MMIO, atualiza um livro de ofertas simplificado em VHDL, executa uma regra de decisão gerada a partir de MATLAB e retorna uma resposta ao ARM. A arquitetura demonstra viabilidade de co-projeto hardware/software em uma plataforma de baixo custo.

O sistema contém quatro superfícies principais. A primeira é a superfície de dados, formada por FAST e TCP. A segunda é a superfície software/hardware, formada por `FpgaSharedStream` e pelos registradores da ponte. A terceira é a superfície de processamento em hardware, formada por livro de ofertas e estratégia gerada. A quarta é a superfície de validação, formada por testes C++, testbenches VHDL e comandos Make.

O sistema não cobre negociação real, risco, roteamento de ordens, autenticação, *market replay* real, multicast, timestamping de precisão ou comparação estatística com dados de bolsa. Essas ausências delimitam o protótipo, mas também definem trabalhos futuros claros.

6.2 Modelagem do sistema

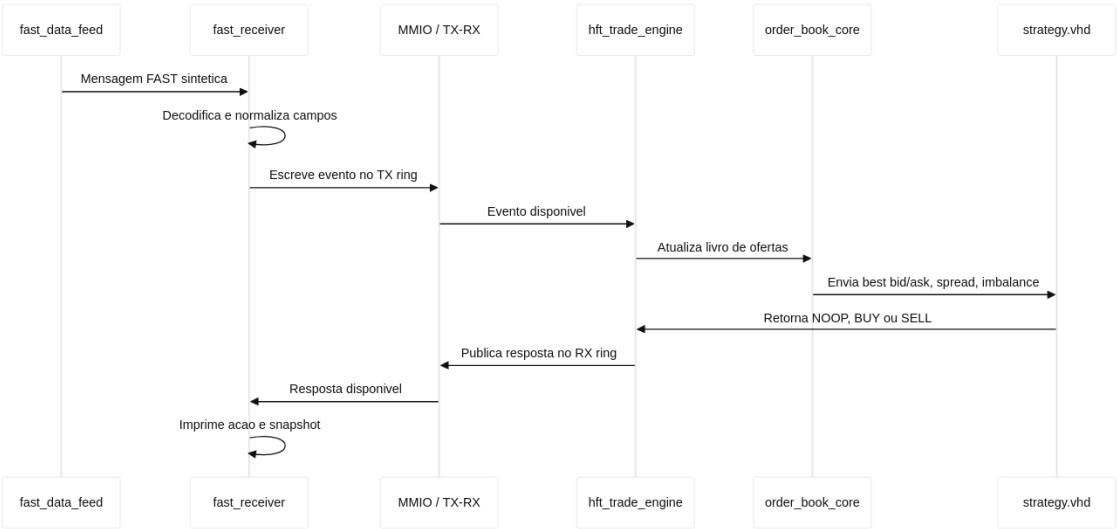
6.2.1 Modelo de componentes

A Figura 5 representa o fluxo de dados implementado. O feed sintético opera no domínio de software; o receptor atua como adaptador entre protocolo financeiro e protocolo de hardware; a ponte converte acessos MMIO em streams internos; o livro de ofertas mantém estado; a estratégia converte sinais de mercado em ação.

Os requisitos funcionais implementados são:

- gerar mensagens sintéticas de mercado com símbolos e lados conhecidos;
- decodificar mensagens FAST e recuperar campos de interesse;
- habilitar ou desabilitar caminho FPGA por variável de ambiente;
- enviar eventos ao FPGA por fila TX;

Figura 5 – Fluxo de dados do protótipo



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

- manter livro de ofertas por símbolo e lado;
- calcular melhor compra, melhor venda, *spread* e desbalanceamento;
- gerar ação NOOP, BUY ou SELL;
- disponibilizar resposta ao software por fila RX;
- validar o contrato por testes automatizados.

Os requisitos não funcionais priorizados são determinismo, rastreabilidade do contrato, simplicidade de reprodução e baixo custo. Requisitos ainda não atendidos incluem medição precisa de latência, robustez contra falhas de hardware em campo, segurança operacional, integração com dados reais e análise formal de temporização pós-síntese.

6.2.2 Modelo de estados das filas

As filas circulares podem ser compreendidas por três estados observáveis: vazia, parcialmente ocupada e cheia. A fila vazia ocorre quando cabeça e cauda possuem o mesmo valor. A fila parcialmente ocupada ocorre quando a cabeça está à frente da cauda em aritmética modular. A fila cheia ocorre quando avançar a cabeça alcançaria a cauda. Essa definição é usada tanto no software quanto no hardware.

Esse modelo evita a necessidade de um contador compartilhado de ocupação. Em um ambiente produtor-consumidor, contador compartilhado exigiria atualização por ambos os lados e poderia introduzir condição de corrida. Com ponteiros separados, cada lado escreve apenas os ponteiros sob sua responsabilidade. O ARM escreve `TX_HEAD` e `RX_TAIL`; o FPGA escreve `TX_TAIL` e `RX_HEAD`. A leitura cruzada é suficiente para inferir disponibilidade.

6.2.3 Modelo de dados em ponto fixo

O preço é representado como inteiro sem sinal escalado por 10^4. Assim, um preço decimal de 185,20 é representado como 1.852.000. Essa escolha facilita comparação e subtração em hardware. Também evita sintetizar unidades de ponto flutuante, que consumiriam mais recursos e tornariam o projeto desnecessariamente complexo.

A limitação é que a escala é fixa. Se o instrumento negociado exigir mais casas decimais ou valores muito altos, a representação precisará ser revista. Para os símbolos sintéticos do feed, a faixa cabe em 32 bits. Em um sistema real, a escala deveria ser definida a partir do *tick size* do mercado e da faixa máxima de preço esperada.

6.2.4 Modelo de registradores

A ponte MMIO expõe identificadores, controle, estado, ponteiros e memórias de slots. O registrador `MAGIC` contém `0x48465431`, correspondente a `HFT1`, e permite ao software verificar se a base física está correta. `VERSION` indica a versão do protocolo. `STATUS` informa disponibilidade de envio, fila TX cheia, RX com dados e RX cheia.

Tabela 3 – Mapa de registradores principal da ponte ARM–FPGA

Offset	Nome	Função
`0x000`	`MAGIC`	Assinatura `HFT1`.
`0x004`	`VERSION`	Versão do protocolo.
`0x008`	`CTRL`	Controle, incluindo reset suave de ponteiros.
`0x00C`	`STATUS`	Bits de fila: pode enviar, TX cheia, RX com dados e RX cheia.
`0x010`	`TX_HEAD`	Ponteiro publicado pelo ARM para TX.
`0x014`	`TX_TAIL`	Ponteiro publicado pelo FPGA para TX.
`0x018`	`RX_HEAD`	Ponteiro publicado pelo FPGA para RX.
`0x01C`	`RX_TAIL`	Ponteiro publicado pelo ARM para RX.
`0x020`	`TX_DEPTH`	Profundidade da fila TX.
`0x024`	`RX_DEPTH`	Profundidade da fila RX.
`0x028`	`SLOT_WORDS`	Quantidade de palavras por slot.
`0x100`	`TX_SLOTS`	Base da memória de slots TX.
dinâmico	`RX_SLOTS`	Base calculada por `0x100 + DEPTH * SLOT_WORDS * 4`.

Fonte: Autoria própria (2026).

O mapa de registradores também funciona como mecanismo de versionamento prático. Em sistemas embarcados, é comum que software e bitstream sejam atualizados em momentos diferentes. Se o software espera slots de oito palavras e o hardware expõe quatro, a falha não é necessariamente visível como erro de compilação; ela aparece como corrupção de dados em tempo de execução. Por isso, a leitura de `SLOT_WORDS`, `TX_DEPTH` e `RX_DEPTH` é parte do resultado, não apenas uma conveniência.

O registrador `CTRL` permite reinicialização lógica dos ponteiros, o que auxilia em bring-up. Sem esse mecanismo, uma execução interrompida poderia deixar cabeça e cauda em posições antigas, obrigando reprogramação da FPGA ou reset completo da placa. A existência de reset suave simplifica ciclos de teste durante desenvolvimento. A limitação é que o reset lógico não substitui uma política completa de recuperação de falhas: se o hardware estiver em estado inválido por reset de clock, erro de temporização ou ponte desabilitada, a solução ainda depende de diagnóstico no nível do sistema.

6.3 Apresentação do sistema

Em execução software-only, o receptor imprime linhas contendo sequência, símbolo, lado, preço e quantidade. Com FPGA habilitado, ele imprime também respostas no formato `[FPGA->ARM]`, contendo ação e instantâneo de livro. A saída esperada inclui campos como `best_bid_px_1e4`, `best_ask_px_1e4`, `spread_1e4` e `imbalance`.

A interface do protótipo é textual porque o foco está na cadeia de dados e não em uma aplicação gráfica. Para fins de monografia, a Figura 6 sugere uma figura com duas janelas de terminal: uma executando `fast_data_feed` e outra executando `fast_receiver` com ou sem `HFT_FPGA_MMIO_BASE`.

Figura 6 – Execução prevista do gerador e receptor

```
seq=40 sym=MSFT side=sell price=415.56 qty=4539 (24 bytes)
seq=41 sym=NVDA side=buy price=874.23 qty=1638 (23 bytes)
seq=42 sym=TSLA side=sell price=175.6 qty=4697 (24 bytes)
seq=43 sym=MSFT side=sell price=415.01 qty=374 (24 bytes)
seq=44 sym=NVDA side=buy price=874.92 qty=2322 (23 bytes)
seq=45 sym=GOOGL side=sell price=170.8 qty=1783 (25 bytes)
seq=46 sym=TSLA side=sell price=175.16 qty=1162 (24 bytes)
seq=47 sym=NVDA side=sell price=875.37 qty=1262 (24 bytes)
seq=48 sym=NVDA side=sell price=875.43 qty=4675 (24 bytes)
seq=49 sym=TSLA side=sell price=175.59 qty=301 (24 bytes)
seq=50 sym=AAPL side=sell price=185.75 qty=1728 (18 bytes)
seq=51 sym=NVDA side=sell price=875.5 qty=3361 (18 bytes)
seq=52 sym=TSLA side=sell price=175.46 qty=2611 (24 bytes)
seq=53 sym=GOOGL side=buy price=169.42 qty=4153 (24 bytes)
seq=54 sym=TSLA side=buy price=174.65 qty=293 (23 bytes)
seq=55 sym=NVDA side=buy price=874.39 qty=2621 (23 bytes)
seq=56 sym=TSLA side=sell price=175.7 qty=939 (24 bytes)
seq=57 sym=GOOGL side=sell price=170.69 qty=708 (25 bytes)
seq=58 sym=MSFT side=buy price=414.38 qty=1924 (23 bytes)
seq=59 sym=AAPL side=sell price=185.12 qty=3921 (24 bytes)
seq=60 sym=NVDA side=sell price=875.07 qty=4902 (24 bytes)
```

```
[FPGA->ARM] seq=53 action=BUY best_bid_px_1e4=1699900 best_bid_qty=2945 best_ask_px_1
e4=1700100 best_ask_qty=1852 spread_1e4=200 imbalance=1093
seq=54 sym=TSLA side=buy price=174.65 qty=293
[FPGA->ARM] seq=54 action=BUY best_bid_px_1e4=1749900 best_bid_qty=2836 best_ask_px_1
e4=1750100 best_ask_qty=681 spread_1e4=200 imbalance=2155
seq=55 sym=NVDA side=buy price=874.39 qty=2621
[FPGA->ARM] seq=55 action=BUY best_bid_px_1e4=8749800 best_bid_qty=3282 best_ask_px_1
e4=8750100 best_ask_qty=2259 spread_1e4=300 imbalance=1023
seq=56 sym=TSLA side=sell price=175.7 qty=939
[FPGA->ARM] seq=56 action=BUY best_bid_px_1e4=1749900 best_bid_qty=2836 best_ask_px_1
e4=1750100 best_ask_qty=681 spread_1e4=200 imbalance=2155
seq=57 sym=GOOGL side=sell price=170.69 qty=708
[FPGA->ARM] seq=57 action=BUY best_bid_px_1e4=1699900 best_bid_qty=2945 best_ask_px_1
e4=1700100 best_ask_qty=1852 spread_1e4=200 imbalance=1093
seq=58 sym=MSFT side=buy price=414.38 qty=1924
[FPGA->ARM] seq=58 action=BUY best_bid_px_1e4=4149900 best_bid_qty=2555 best_ask_px_1
e4=4150100 best_ask_qty=374 spread_1e4=200 imbalance=2181
seq=59 sym=AAPL side=sell price=185.12 qty=3921
[FPGA->ARM] seq=59 action=NOOP best_bid_px_1e4=1849900 best_bid_qty=939 best_ask_px_1
e4=1850300 best_ask_qty=818 spread_1e4=400 imbalance=121
seq=60 sym=NVDA side=sell price=875.07 qty=4902
```

Fonte: Autoria própria (2026).

O modo textual também ajuda a depuração. Cada linha impressa pelo receptor conserva a sequência do evento original, e cada resposta FPGA ecoa essa sequência. Isso permite verificar se respostas estão atrasadas, perdidas ou associadas ao evento errado. Em versões futuras, a saída textual poderia ser complementada por arquivo CSV ou logs estruturados, mas para o estágio atual a impressão direta é suficiente.

6.3.1 Execução funcional na DE10-Nano

Foi realizada uma execução funcional na DE10-Nano acessada por SSH. O receptor foi iniciado com `HFT_FPGA_MMIO_BASE=0xFF200000`, habilitando o caminho FPGA. A inicialização indicou leitura correta da ponte MMIO, com `magic=0x48465431`, `version=1`, `tx_depth=64`, `rx_depth=64`, `slot_words=8` e `rx_base=0x900`. Esses valores confirmam que o software encontrou o periférico esperado, reconheceu a geometria do contrato e ativou o modo novo da interface.

Em seguida, o receptor conectou-se ao feed local em `127.0.0.1:9001`. O gerador `fast_data_feed` publicou mensagens sintéticas FAST com símbolos AAPL, MSFT, NVDA, GOOGL e TSLA. A partir da sequência 26, o servidor indicou cliente conectado e o receptor passou a imprimir pares de linhas: uma linha com o evento decodificado e outra com a resposta `[FPGA->ARM]`. Foram observadas respostas BUY e NOOP contendo melhor compra, melhor venda, quantidades, *spread* e desbalanceamento.

Como exemplo, para o evento `seq=26`, símbolo TSLA e lado de venda, a resposta retornou ação BUY, melhor compra `1749900`, melhor venda `1750100`, *spread* `200` e desbalanceamento `2155`. Para eventos do símbolo AAPL, como `seq=36`, a resposta retornou NOOP com *spread* `400` e desbalanceamento `121`. Esses resultados são coerentes com a regra de estratégia: BUY ocorre quando o desbalanceamento comprador supera o limiar e NOOP ocorre quando o sinal não atinge condição de compra ou venda.

O trecho apresentado no Listagem 1 evidencia dois comportamentos relevantes. Primeiro, a leitura do cabeçalho da ponte confirma que o software acessou a região MMIO correta. Segundo, as respostas `[FPGA->ARM]` mostram que o FPGA retornou ações e campos derivados do livro de ofertas para eventos reais do feed sintético. O caso TSLA e o caso NVDA resultaram em BUY devido a desbalanceamentos positivos superiores ao limiar; o caso AAPL resultou em NOOP, pois o desbalanceamento de 121 não satisfaz a condição de compra ou venda.

Tabela 4 – Resumo da execução funcional na DE10-Nano

Item observado	Evidência
Ponte MMIO reconhecida	`magic=0x48465431`, `version=1`, `slot_words=8`.
Geometria das filas	`tx_depth=64`, `rx_depth=64`, `rx_base=0x900`.
Feed conectado	Receptor conectado a `127.0.0.1:9001`; servidor indica cliente conectado.
Eventos processados	Sequências 26 a 52 recebidas e respondidas antes da interrupção manual.
Ações observadas	BUY para desbalanceamentos positivos elevados e NOOP para AAPL com desbalanceamento baixo.
Campos de livro retornados	Melhor compra, melhor venda, quantidades, *spread* e desbalanceamento impressos pelo ARM.

Fonte: Autoria própria (2026).

Listagem 1 – Trecho da saída do receptor com respostas FPGA–ARM

```
FPGA MMIO bridge enabled: base=0xff200000 span=0x2000
magic=0x48465431 version=1 tx_depth=64 rx_depth=64
slot_words=8 rx_base=0x900 mode=new
Connected to 127.0.0.1:9001

seq=26 sym=TSLA side=sell price=175.17 qty=1547
[FPGA->ARM] seq=26 action=BUY
  best_bid_px_1e4=1749900 best_bid_qty=2836
  best_ask_px_1e4=1750100 best_ask_qty=681
  spread_1e4=200 imbalance=2155

seq=27 sym=NVDA side=sell price=875.13 qty=3861
[FPGA->ARM] seq=27 action=BUY
  best_bid_px_1e4=8749800 best_bid_qty=3282
  best_ask_px_1e4=8750100 best_ask_qty=2259
  spread_1e4=300 imbalance=1023

seq=36 sym=AAPL side=sell price=185.75 qty=1795
[FPGA->ARM] seq=36 action=NOOP
  best_bid_px_1e4=1849900 best_bid_qty=939
  best_ask_px_1e4=1850300 best_ask_qty=818
  spread_1e4=400 imbalance=121
```

Fonte: Execução funcional na DE10-Nano realizada pelos autores..

Esse ensaio deve ser interpretado como evidência funcional de integração em placa. Ele demonstra que o caminho feed–receptor–MMIO–FPGA–MMIO–receptor opera e que a estratégia gerada está produzindo respostas consistentes com os sinais do livro. Entretanto, ele ainda não mede latência, vazão máxima ou uso de recursos. Essas métricas exigem timestamps, contadores de ciclo e coleta sistemática, conforme planejado na metodologia.

6.4 Implementação do sistema

6.4.1 Wrapper MMIO em C++

O arquivo `fpga_shared_stream.h` implementa a classe `FpgaSharedStream`, responsável por abrir `/dev/mem` ou um arquivo de teste, mapear a região física, ler o cabeçalho do dispositivo e operar filas. A classe também possui compatibilidade com um layout legado, porém prioriza o cabeçalho novo com `MAGIC`, `VERSION` e geometria.

O cálculo dinâmico é relevante porque a base RX depende de profundidade e largura de slot. Se a implementação usasse um valor fixo, qualquer alteração em `G_DEPTH` ou `G_SLOT_WORDS` poderia fazer o ARM ler a região errada.

Listagem 2 – Cálculo dinâmico da base RX no wrapper C++

```
uint32_t TxBase() const { return kRingBase; }

uint32_t RxBase() const {
  return TxBase() + tx_depth_ * slot_words_ * sizeof(uint32_t);
}
```

Fonte: Adaptado do código-fonte do projeto.

O wrapper também protege contra geometria incompatível. Após mapear a região, ele lê profundidade e largura de slot. Se os valores forem nulos, excessivos ou incompatíveis com o *span* mapeado, a abertura falha. Essa verificação evita que o receptor escreva fora da janela MMIO ou interprete memória aleatória como slots. Em hardware real, esse tipo de proteção é importante porque um endereço base incorreto pode apontar para outro periférico ou para uma região sem resposta.

6.4.2 Receptor FAST e empacotamento de eventos

O receptor decodifica cada entrada de mercado e monta o quadro enviado ao FPGA. A conversão de preço usa escala 10^4, e o mapeamento de símbolo remove strings do caminho de hardware. O trecho a seguir resume o empacotamento:

Listagem 3 – Empacotamento do quadro ARM–FPGA

```
frame.word0 = entry.get_SeqNo().value();
frame.word1 = symbol_id;
frame.word2 = price_to_fixed_1e4(entry.get_Price());
frame.word3 = entry.get_Qty().value();
frame.word4 = kEventUpsertLevel;
frame.word5 = parse_side_code(entry.get_Side().c_str());
frame.word6 = 0;
frame.word7 = 0;
```

Fonte: Adaptado do código-fonte do projeto.

A vantagem é a clareza do contrato. A limitação é que eventos reais possuem mais campos, como tipo de atualização detalhado, identificadores de ordem, timestamp de mercado e informações de sessão.

O receptor também contém lógica de varredura opcional da ponte, controlada por variável de ambiente. Essa lógica tenta localizar uma base compatível dentro da janela leve HPS–FPGA, procurando o cabeçalho esperado. O recurso não é parte do caminho crítico, mas é útil em laboratório, pois offsets do Platform Designer podem mudar durante integração.

6.4.3 Ponte compartilhada em VHDL

O módulo `arm_fpga_shared_stream_bridge.vhd` implementa registradores, memórias TX/RX e handshakes. A função `f_slot_word_set` escreve uma palavra de 32 bits em uma lane do slot, preservando ordenação little-endian por índice de palavra. O consumo TX ocorre quando `cmd_valid_s` e `cmd_ready_i` estão ativos. A captura RX ocorre quando `rsp_valid_i` está ativo e a fila RX não está cheia.

Essa ponte tem duas vantagens: desacopla o tempo de escrita do ARM do tempo de processamento do FPGA e torna o contrato testável por simulação. O principal trade-off é que MMIO com polling não é a forma mais eficiente de transferir grandes volumes; uma versão futura poderia explorar DMA ou streaming direto de rede.

O bloco também expõe leitura da memória TX e RX. A leitura de TX ajuda a depurar o que o ARM escreveu. A escrita de RX por MMIO aparece como caminho opcional de depuração em simulação e bring-up. Em um produto final, a superfície de debug poderia ser reduzida para diminuir área ou evitar escritas indevidas. No protótipo, a observabilidade tem maior valor do que a minimização absoluta do mapa.

6.4.4 Livro de ofertas

O `order_book_core.vhd` mantém quatro matrizes: preços e quantidades de compra, preços e quantidades de venda. O procedimento `p_apply_level_update` trata busca por preço existente, remoção, inserção ordenada e deslocamento. A cada evento, o bloco produz um snapshot. O circuito rejeita livros cruzados, evitando que uma compra seja inserida acima ou igual à melhor venda, ou que uma venda seja inserida abaixo ou igual à melhor compra.

O comportamento do livro pode ser dividido em casos. Se o evento é reset, ambos os lados do símbolo são limpos. Se o preço já existe e a quantidade é positiva, a quantidade é atualizada no próprio nível. Se o preço já existe e a atualização é remoção ou quantidade zero, o nível é removido e os níveis seguintes são compactados. Se o preço não existe e a quantidade é positiva, o nível é inserido na posição que preserva ordenação. Se o livro está cheio e o novo nível é pior que todos os níveis armazenados, o evento é ignorado.

Essa política é compatível com um livro de profundidade limitada. O sistema não precisa preservar níveis além de `G_BOOK_DEPTH`, pois a estratégia atual só usa o topo do livro. Em estratégias futuras que usem profundidade acumulada, essa decisão precisará ser reavaliada.

6.4.5 Estratégia MATLAB HDL Coder

A função MATLAB `strategy.m` retorna BUY se o desbalanceamento é positivo acima do limiar e o *spread* está aceitável, SELL se o desbalanceamento é negativo abaixo do limiar,

Figura 7 – Pipeline lógico de atualização do livro

Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

e NOOP caso contrário. Ela é compatível com HDL Coder por usar tipos inteiros fixos e não depender de estruturas dinâmicas. O wrapper VHDL `generated_strategy_core.vhd` encapsula o bloco gerado e monta o quadro de resposta.

O wrapper da estratégia mantém protocolo de *valid/ready*. Quando já existe resposta pendente e o consumidor não está pronto, o bloco não aceita novo snapshot. Essa contrapressão impede perda silenciosa de resposta. A decisão gerada é combinacional, mas a resposta é registrada, o que estabiliza o dado entregue à fila RX.

6.4.6 Projeto Quartus e componente customizado

O projeto Quartus em `quartus/de10_nano_hft` configura o dispositivo Cyclone V e inclui o sistema Platform Designer. O arquivo `hft.qsys` representa a conexão entre HPS, clock e componente customizado. O arquivo `hft_trade_engine_avalon_mm_hw.tcl` registra o motor HFT como IP, define parâmetros e declara a interface Avalon-MM. Com isso, o bloco pode ser instanciado no Platform Designer como um periférico acessível pelo HPS.

Essa integração é essencial para sair da simulação e chegar à placa. O software assume que a base física corresponde à janela leve HPS–FPGA, usualmente `0xFF200000`, somada ao offset atribuído ao componente. Se o offset mudar, o valor de `HFT_FPGA_MMIO_BASE` deve mudar também. Essa dependência é documentada porque muitos erros de bring-up em SoC FPGA ocorrem por endereço incorreto, ponte desabilitada ou reset mal conectado.

6.4.7 Fluxo de build e dependências

O fluxo de build resolve dependências em camadas. Primeiro, compila ou instala mFAST. Depois, gera tipos C++ do template FAST. Em seguida, constrói os executáveis de feed e receptor. Para hardware, gera VHDL da estratégia MATLAB, executa simulações com GHDL e integra os arquivos no Quartus. Para ARM, usa cross-build com sysroot extraído da placa. O objetivo desse encadeamento é evitar que uma etapa use artefatos antigos sem perceber.

O alvo `make build` é mais amplo que `make check`. Ele inclui geração HDL, testes, cross-build e Quartus. Em uma rotina de desenvolvimento, mudanças pequenas no wrapper C++ podem ser validadas com `make cpp-test`; mudanças de RTL com `make vhdl-test-*`; mudanças de contrato exigem múltiplos alvos. Essa granularidade reduz tempo de feedback.

O uso de Docker no fluxo de software e simulação tem impacto direto sobre a reprodutibilidade. A biblioteca mFAST e as ferramentas C++ podem ter comportamento sensível a versões de compilador e dependências. Ao fixar o ambiente, a plataforma reduz a chance de que um teste passe em uma máquina e falhe em outra por diferença de pacote. No fluxo FPGA, essa estabilidade é mais difícil porque Quartus e MATLAB possuem licenças e imagens próprias; ainda assim, o Makefile explicita os comandos esperados e documenta a dependência.

6.4.8 Critérios de aceitação funcional

Com base nos testes e no contrato, foram definidos critérios de aceitação para considerar o protótipo funcional. Esses critérios são apresentados no Quadro 5. Eles servem como ponte entre implementação e avaliação acadêmica, pois transformam requisitos de engenharia em observações verificáveis.

Quadro 5 – Critérios de aceitação funcional

Critério	Evidência esperada
Identificação da ponte	Software lê `MAGIC=0x48465431`, versão e geometria compatível antes de enviar eventos.
Publicação TX correta	Payload é escrito no slot antes do avanço de `TX_HEAD`.
Consumo TX correto	FPGA observa diferença entre `TX_HEAD` e `TX_TAIL`, consome slot e avança cauda.
Resposta RX correta	FPGA escreve resposta completa antes de avançar `RX_HEAD`; ARM lê slot antes de avançar `RX_TAIL`.
Livro ordenado	Compras permanecem em ordem decrescente e vendas em ordem crescente.
Proteção de estado inválido	Atualizações que cruzariam o livro são rejeitadas ou não geram ordem.
Estratégia verificável	BUY, SELL e NOOP aparecem em cenários controlados de desbalanceamento e *spread*.
Integração Avalon	Leituras e escritas MMIO completam sem travar `waitrequest`.

Fonte: Autoria própria (2026).

6.5 Discussões

Os testes automatizados indicam que o contrato funcional está coerente entre C++ e VHDL. O teste C++ valida a fila com profundidade 4, demonstrando que apenas 3 posições são utilizáveis, como esperado para a convenção `next(head) == tail`. O testbench do motor completo escreve três eventos MMIO e verifica respostas NOOP, BUY e SELL, incluindo topo de livro e desbalanceamento. O testbench do livro valida rejeição de livro cruzado, caso importante para evitar decisões sobre estado inconsistente.

O principal resultado técnico é a estabilização da interface e do fluxo de geração da estratégia. Em projetos hardware/software, erros de integração frequentemente surgem de detalhes aparentemente pequenos: base RX fixa, largura de slot divergente, publicação de ponteiro antes do payload, leitura após avanço de ponteiro ou inversão de ordem de palavras. A plataforma trata esses pontos com documentação e testes, o que aumenta a confiabilidade do ambiente e permite que a estratégia MATLAB seja alterada sem redesenhar toda a infraestrutura.

6.5.1 Interpretação dos testes

O teste `fpga_shared_stream_test` é relevante porque exercita o wrapper sem precisar de hardware real. Ele cria um arquivo temporário, escreve os registradores esperados e permite que a classe C++ opere como se estivesse mapeando MMIO. Esse teste verifica fila cheia com profundidade 4, o que confirma a regra de capacidade `DEPTH - 1`. Também verifica leitura de resposta e avanço de `RX_TAIL`.

O testbench `tb_order_book_core` testa casos fundamentais do livro: inserção de compra, inserção de venda, melhoria de melhor compra, rejeição de venda cruzada, rejeição de compra cruzada e remoção. Esses casos são suficientes para validar os invariantes mais importantes da estrutura simplificada. O testbench do motor completo confirma que eventos escritos por MMIO atravessam a ponte, chegam ao livro, acionam a estratégia e retornam pela RX.

O testbench Avalon-MM tem foco diferente: ele não valida a estratégia financeira, mas a robustez da interface de barramento. Ao verificar que `waitrequest` não fica preso durante reset e que uma transação completa ou retorna sentinela, ele cobre um risco prático de integração com Linux embarcado. Isso mostra que os testes não são redundantes; cada um protege uma camada diferente.

6.5.2 Riscos técnicos remanescentes

Mesmo com os testes atuais, alguns riscos permanecem. Simulação VHDL não substitui análise de temporização pós-síntese. Um circuito funcional em GHDL pode falhar em frequência se o caminho combinacional for longo. O procedimento de atualização do livro possui laços sobre profundidade e pode limitar frequência quando `G_BOOK_DEPTH` aumentar. Portanto, medições futuras devem incluir *timing report* do Quartus.

Outro risco é coerência de memória no caminho MMIO em hardware real. O wrapper usa acessos voláteis e barreiras, mas o comportamento final depende também de mapeamento do dispositivo, atributos de cache e ponte HPS. Em ambientes embarcados, regiões MMIO devem ser mapeadas como dispositivo, não como memória cacheável. O uso de `/dev/mem` com `O_SYNC` é apropriado para bring-up, mas uma solução de produção exigiria driver dedicado.

Há também risco de interpretação dos resultados. Como o feed é sintético e roda sobre TCP, qualquer métrica de latência obtida nesse estágio incluiria características do ambiente de teste que não representam uma bolsa real. Isso não invalida o protótipo, mas impõe cuidado na redação das conclusões. O sistema demonstra o caminho de dados e a integração; a afirmação de ganho de desempenho exige experimento comparativo controlado.

O Quadro 6 resume riscos remanescentes e ações de mitigação recomendadas. Essa tabela ajuda a transformar limitações em roteiro de continuidade.

6.5.3 Espaço reservado para resultados quantitativos

Para transformar os resultados funcionais em resultados quantitativos, serão instrumentados três pontos: entrada do evento no receptor, publicação do quadro TX e leitura da resposta RX. No FPGA, será incluído um contador de ciclos no momento de consumo TX e no momento

Quadro 6 – Riscos técnicos remanescentes e mitigação

Risco	Mitigação recomendada
Caminho crítico longo no livro	Coletar relatório de temporização, inserir pipeline e avaliar profundidades menores ou memórias dedicadas.
MMIO limitando vazão	Comparar polling com interrupções ou DMA; medir eventos por segundo antes de saturação.
Endereço físico incorreto	Usar leitura de `MAGIC`, varredura de base e documentação do offset do Platform Designer.
Regeneração HLS incompatível	Manter wrapper estável, versionar portas geradas e executar testbench da estratégia após cada geração.
Feed pouco realista	Adicionar replay de dados históricos, bursts, múltiplos símbolos simultâneos e timestamps.
Ausência de driver dedicado	Evoluir de `/dev/mem` para módulo de kernel ou UIO com atributos de memória controlados.
Comparação de desempenho incompleta	Implementar versão software-only equivalente do livro e coletar distribuição de latência.

Fonte: Autoria própria (2026).

de publicação RX. Com esses dados, será possível separar latência de software, latência de transferência e latência de processamento em hardware.

A vazão pode ser avaliada aumentando a taxa do feed e observando quando `bridge.Send` começa a falhar por fila cheia. Também é possível medir ocupação média de TX e RX por leitura periódica de ponteiros. Esses experimentos permitiriam preencher os gráficos sugeridos neste capítulo e comparar configurações de profundidade de fila, profundidade do livro e limiares de estratégia.

Os resultados ainda são insuficientes para afirmar ganho quantitativo de desempenho. Para isso, serão necessários experimentos com timestamp antes e depois de cada etapa, coleta de distribuições, comparação com versão software-only equivalente e relatórios de síntese com frequência máxima e uso de recursos. As metas de referência são: jitter interno inferior a $1\ \mu$s, vazão mínima de 100.000 mensagens por segundo, speedup mínimo de 5 vezes em relação a uma versão software-only equivalente e ocupação de recursos compatível com a Cyclone V da DE10-Nano. O Gráfico 2 indica o tipo de gráfico reservado para essa etapa.

Também deve ser medida a produtividade do fluxo HLS. Uma forma prática é implementar uma segunda estratégia MATLAB simples, por exemplo alterando os limiares de *spread* e desbalanceamento, e registrar o tempo necessário desde a edição de `strategy.m` até a aprovação do testbench. Caso essa alteração não exija mudanças no barramento, no livro de ofertas ou no wrapper Avalon-MM, haverá evidência de que o objetivo de facilitar a criação de estratégias foi atendido.

Gráfico 2 – Vazão e ocupação de filas sugeridas para avaliação futura

[GRÁFICO 2 – Sugestão: eventos por segundo suportados antes de fila TX cheia e evolução de ocupação das filas TX/RX para diferentes taxas de chegada.]

Fonte: Autoria própria (2026).

Gráfico 3 – Uso de recursos FPGA sugerido para relatório pós-síntese

[GRÁFICO 3 – Sugestão: porcentagem de LUTs, registradores, memória interna e multiplicadores usados por ponte, livro de ofertas e estratégia.]

Fonte: Autoria própria (2026).

7 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a documentação de uma plataforma de HFT de baixo custo baseada em co-projeto ARM–FPGA, com foco principal na criação de estratégias de trading em MATLAB e geração de hardware por meio do MATLAB HDL Coder. A infraestrutura desenvolvida inclui gerador FAST sintético, receptor C++ com mFAST, ponte MMIO por filas circulares, livro de ofertas simplificado em VHDL, wrapper para estratégia gerada por HLS, wrapper Avalon-MM e fluxo de build/teste baseado em Make, Docker, CMake, GHDL, MATLAB e Quartus.

O objetivo geral foi atendido no nível funcional: o sistema permite isolar a estratégia como função MATLAB, gerar VHDL com HDL Coder, integrar o bloco gerado ao motor de decisão e executar o caminho de dados entre software e FPGA. O trabalho também consolidou o contrato entre C++ e VHDL, enfatizando registradores, ponteiros, capacidade efetiva de filas, cálculo dinâmico da base RX, largura de oito palavras por quadro e ordem de empacotamento. Esse contrato é uma contribuição de engenharia essencial, pois mantém estável a infraestrutura sobre a qual novas estratégias podem ser testadas.

Entre as vantagens observadas, destacam-se a modularidade da arquitetura, a separação adequada entre software flexível e hardware determinístico, a possibilidade de alterar a regra de estratégia em MATLAB e regenerá-la por HDL Coder, e a existência de testes em diferentes níveis. A abordagem por filas circulares é simples, dispensa bloqueios entre produtor e consumidor e facilita a depuração. A modelagem do livro por profundidade fixa favorece síntese em FPGA e evita estruturas dinâmicas incompatíveis com RTL.

Os objetivos específicos também foram contemplados. A plataforma foi organizada para que a estratégia seja o principal bloco substituível pelo usuário. O estudo teórico apresentou os conceitos necessários de HFT, livro de ofertas, FPGA, HLS, MATLAB HDL Coder, FAST, MMIO, Avalon-MM e métricas de desempenho. A documentação do contrato ARM–FPGA foi incorporada ao texto e detalhada em apêndice. Os módulos de software, hardware e infraestrutura foram descritos com suas funções, vantagens, limitações e integração. Por fim, a validação funcional foi organizada em uma matriz que relaciona testes a riscos técnicos.

Do ponto de vista acadêmico, a contribuição central não é uma estratégia de negociação lucrativa, mas um fluxo de criação de estratégias para FPGA. Essa distinção é importante. Estratégias de HFT reais dependem de dados proprietários, controle de risco, colocação de ordens, posição, custos e conformidade. A plataforma, por sua vez, demonstra como uma aplicação sensível a latência pode ser decomposta em software e hardware, como representar eventos em ponto fixo, como preservar ordenação em uma fila circular e como usar HLS para reduzir o esforço de implementação do bloco decisório.

As limitações também são claras. O feed é sintético e usa TCP, não dados reais de bolsa. O livro de ofertas é por nível agregado, não por ordem individual. A estratégia é didática e não considera posição, risco, custos, execução ou conformidade regulatória. As medições quantita-

tivas de latência, vazão e uso de recursos ainda precisam ser realizadas com instrumentação adequada. Além disso, MMIO com polling é suficiente para o protótipo desenvolvido, mas pode não ser adequado para taxas profissionais de dados de mercado.

Outra limitação é o estágio das métricas. A execução funcional em placa confirma que o caminho feed–ARM–FPGA–ARM opera, mas a avaliação de desempenho ainda precisa de instrumentação consistente. Para reivindicar desempenho em placa, é necessário coletar timestamps, contadores de ciclo, vazão máxima, uso de recursos e frequência de operação. Essa cautela evita transformar uma validação funcional em afirmação quantitativa não comprovada.

Como trabalhos futuros, recomenda-se: incluir timestamps no ARM e no FPGA para decompor latência por etapa; coletar relatórios de síntese e temporização do Quartus; comparar a versão FPGA com uma versão software-only equivalente do livro de ofertas; experimentar DMA ou integração com recepção de rede em hardware; ampliar o modelo de eventos para cancelamentos, execuções e múltiplos níveis; validar o fluxo em DE10-Nano com bitstream programado; e estudar estratégias mais realistas com controle de risco. Também seria relevante automatizar a geração de figuras e gráficos a partir dos resultados de teste, substituindo os placeholders indicados neste texto por evidências experimentais.

A evolução natural do projeto pode ser dividida em três frentes. A primeira é experimental: medir latência, vazão, recursos FPGA e produtividade de alteração de estratégias MATLAB. A segunda é arquitetural: substituir polling por DMA ou interrupções, aproximar a recepção de rede do FPGA e modularizar a estratégia para múltiplos algoritmos. A terceira é financeira: enriquecer o modelo de eventos, incluir controle de posição, limites de risco e custos de transação. Essas frentes são complementares e devem ser conduzidas nessa ordem para evitar que uma estratégia mais complexa esconda problemas básicos de infraestrutura.

Conclui-se que a proposta de uma plataforma de HFT de baixo custo orientada à geração de estratégias por MATLAB HDL Coder é viável no nível funcional apresentado. Mesmo sem atingir o escopo de um sistema financeiro real, o protótipo demonstra que uma estratégia escrita em MATLAB pode ser tratada como bloco substituível, gerada para VHDL por HLS e integrada a uma infraestrutura ARM–FPGA com livro de ofertas, filas MMIO e validação cruzada. Esses elementos constituem uma base sólida para a continuidade do projeto, especialmente na coleta de métricas quantitativas e na avaliação de novas estratégias descritas em MATLAB.

REFERÊNCIAS

ALDRIDGE, I. **High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems**. 2. ed. [*S.l.*]: Wiley, 2013.

BOUTROS, A. *et al.* Build fast, trade fast: Fpga-based high-frequency trading using high-level synthesis. *In*: **2017 International Conference on ReConFigurable Computing and FPGAs**. Cancun, Mexico: IEEE, 2017. p. 1–6.

BROGAARD, J.; HENDERSHOTT, T.; RIORDAN, R. High-frequency trading and price discovery. **The Review of Financial Studies**, v. 27, n. 8, p. 2267–2306, 2014.

CONT, R.; STOIKOV, S.; TALREJA, R. A stochastic model for order book dynamics. **Operations Research**, v. 58, n. 3, p. 549–563, 2010.

Docker Inc. **Docker Documentation**. [*S.l.*], 2026. Disponível em: https://docs.docker.com/. Acesso em: 13 abr. 2026.

FIX Trading Community. **FIX Adapted for STreaming Technical Standard**. [*S.l.*], 2006. Disponível em: https://www.fixtrading.org/standards/fast-online/. Acesso em: 13 abr. 2026.

GHDL Project. **GHDL Documentation**. [*S.l.*], 2026. Disponível em: https://ghdl.github.io/ghdl/. Acesso em: 13 abr. 2026.

GOULD, M. D. *et al.* Limit order books. **Quantitative Finance**, v. 13, n. 11, p. 1709–1742, 2013.

HARRIS, D. M.; HARRIS, S. L. **Digital Design and Computer Architecture**. 2. ed. [*S.l.*]: Morgan Kaufmann, 2013.

HASBROUCK, J.; SAAR, G. Low-latency trading. **Journal of Financial Markets**, v. 16, n. 4, p. 646–679, 2013.

HENDERSHOTT, T.; JONES, C. M.; MENKVELD, A. J. Does algorithmic trading improve liquidity? **The Journal of Finance**, v. 66, n. 1, p. 1–33, 2011.

Intel Corporation. **Avalon Interface Specifications**. [*S.l.*], 2024. Disponível em: https://www.intel.com/content/www/us/en/docs/programmable/683091/current/avalon-memory-mapped-interfaces.html. Acesso em: 13 abr. 2026.

Intel Corporation. **Intel Quartus Prime Platform Designer User Guide**. [*S.l.*], 2024. Disponível em: https://www.intel.com/content/www/us/en/docs/programmable/683609/current/platform-designer-system-integration-tool.html. Acesso em: 13 abr. 2026.

Intel Corporation. **Cyclone V FPGA and SoC FPGA Product Brief**. [*S.l.*], 2025. Disponível em: https://cdrdv2-public.intel.com/853455/cyclone-v-fpgas-and-socs-product-brief.pdf. Acesso em: 13 abr. 2026.

Kitware, Inc. **CMake Toolchains Documentation**. [*S.l.*], 2026. Disponível em: https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html. Acesso em: 13 abr. 2026.

LEBER, C.; GEIB, B.; LITZ, H. High frequency trading acceleration using fpgas. *In*: **2011 21st International Conference on Field Programmable Logic and Applications**. [*S.l.*: *s.n.*], 2011. p. 317–322.

LOCKWOOD, J. W. *et al.* A low-latency library in fpga hardware for high-frequency trading. *In*: **2012 IEEE 20th Annual Symposium on High-Performance Interconnects**. [*S.l.*: *s.n.*], 2012. p. 9–16.

Object Computing, Inc. **mFAST: a C++ FIX/FAST Encoder and Decoder**. [*S.l.*], 2026. Disponível em: https://github.com/objectcomputing/mFAST. Acesso em: 13 abr. 2026.

PATTERSON, D. A.; HENNESSY, J. L. **Computer Organization and Design: The Hardware/Software Interface**. 5. ed. [*S.l.*]: Morgan Kaufmann, 2017.

Terasic Technologies. **DE10-Nano User Manual**. [*S.l.*], 2024. Disponível em: https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=1046. Acesso em: 13 abr. 2026.

The MathWorks, Inc. **Fixed-Point Designer Documentation**. [*S.l.*], 2026. Disponível em: https://www.mathworks.com/help/fixedpoint/. Acesso em: 13 abr. 2026.

The MathWorks, Inc. **HDL Coder Documentation**. [*S.l.*], 2026. Disponível em: https://www.mathworks.com/help/hdlcoder/. Acesso em: 13 abr. 2026.

APÊNDICE A – Contrato de Comunicação ARM–FPGA

Este apêndice consolida o contrato de comunicação utilizado pelo protótipo. Ele complementa a descrição do Capítulo 4 e deve ser mantido alinhado aos arquivos `cpp/src/fpga_shared_stream.h`, `cpp/src/fast_receiver.cpp`, `vhdl/arm_fpga_shared_stream_bridge.vhd` e `docs/arm-fpga-shared-memory-stre`

A.1 Registradores

Tabela 5 – Registradores da interface MMIO

Offset	Nome	Acesso	Dono	Descrição
`0x000`	`MAGIC`	RO	FPGA	Valor `0x48465431`.
`0x004`	`VERSION`	RO	FPGA	Versão do protocolo.
`0x008`	`CTRL`	RW	ARM	Bit 0 reinicia ponteiros.
`0x00C`	`STATUS`	RO	FPGA	Bits de disponibilidade das filas.
`0x010`	`TX_HEAD`	RW	ARM	Publicação de comandos.
`0x014`	`TX_TAIL`	RO	FPGA	Consumo de comandos.
`0x018`	`RX_HEAD`	RO	FPGA	Publicação de respostas.
`0x01C`	`RX_TAIL`	RW	ARM	Consumo de respostas.
`0x020`	`TX_DEPTH`	RO	FPGA	Profundidade da fila TX.
`0x024`	`RX_DEPTH`	RO	FPGA	Profundidade da fila RX.
`0x028`	`SLOT_WORDS`	RO	FPGA	Palavras por quadro.

Fonte: Autoria própria (2026).

A.2 Regras de filas

As filas TX e RX usam a mesma semântica:

- vazia: `head == tail`;
- cheia: `next(head) == tail`;
- capacidade útil: `DEPTH - 1`;
- produtor escreve payload antes de publicar `head`;
- consumidor lê payload antes de publicar `tail`.

Para `DEPTH = 64` e `SLOT_WORDS = 8`, a base TX é `0x100`, cada slot tem 32 bytes e a base RX é `0x900`.

A.3 Formato de evento

O evento enviado do ARM para o FPGA usa oito palavras. A palavra de sequência é preservada para correlação entre entrada e saída. O identificador de símbolo é numérico para

evitar processamento textual em hardware. O preço é escalado por 10^4 para representar casas decimais com inteiros.

Tabela 6 – Formato do quadro ARM–FPGA

Palavra	Campo	Semântica
`word0`	Sequência	Número de sequência da mensagem sintética.
`word1`	Símbolo	Identificador numérico atribuído pelo receptor.
`word2`	Preço	Preço em escala 10^4.
`word3`	Quantidade	Quantidade agregada no nível de preço.
`word4`	Tipo de evento	Atualização/inserção, remoção ou reset.
`word5`	Lado	Compra ou venda.
`word6`	Reserva	Campo reservado para extensão futura.
`word7`	Reserva	Campo reservado para extensão futura.

Fonte: Autoria própria (2026).

A.4 Formato de resposta

O software interpreta a resposta do FPGA como:

- `word0`: sequência;
- `word1`: ação;
- `word2`: melhor compra em escala 10^4;
- `word3`: quantidade na melhor compra;
- `word4`: melhor venda em escala 10^4;
- `word5`: quantidade na melhor venda;
- `word6`: *spread*;
- `word7`: desbalanceamento assinado.

A.5 Checklist de alteração do contrato

Quando o contrato ARM–FPGA for alterado, os seguintes itens devem ser revisados em conjunto:

- mapa de registradores no VHDL e na documentação;
- largura `G_SLOT_WORDS` no VHDL e `kFrameWords` no C++;
- cálculo da base RX no software, nos testbenches e no texto;

- ordem das palavras em `fast_receiver.cpp`;
- decodificação das palavras em `order_book_core.vhd`;
- montagem da resposta em `generated_strategy_core.vhd`;
- testes C++ e VHDL que validam fila cheia, *wrap-around* e resposta;
- comandos e exemplos de execução no README e neste apêndice.

Esse checklist existe porque mudanças pequenas em interfaces binárias podem causar erros difíceis de diagnosticar. Em particular, uma alteração de largura de slot muda a base RX; uma alteração de ordem de palavra pode fazer preço ser interpretado como quantidade; e uma alteração de evento sem atualização do livro pode gerar snapshots inconsistentes.

APÊNDICE B – Comandos de Construção, Teste e Execução

Este apêndice reúne comandos úteis para reproduzir a construção e validação do protótipo.

B.1 Validação local

Listagem 4 – Comandos de teste local

```
make docker-image
make cpp-test
make cpp-smoke
make vhdl-test-all
```

Fonte: Autoria própria (2026).

O alvo `cpp-test` valida o wrapper MMIO com arquivo temporário. O alvo `cpp-smoke` executa o gerador e o receptor em modo software. O alvo `vhdl-test-all` executa testbenches da ponte, livro, estratégia, motor completo e wrapper Avalon-MM.

B.2 Geração da estratégia

Listagem 5 – Geração do bloco de estratégia por HDL Coder

```
make matlab-image
make matlab-hdl-generate
```

Fonte: Autoria própria (2026).

O primeiro comando prepara a imagem esperada para execução do MATLAB em ambiente controlado. O segundo executa o script `matlab/hdl_generator.m`, que chama a geração VHDL da função `strategy.m`. Após essa etapa, recomenda-se executar o testbench da estratégia e o testbench do motor completo, pois uma mudança no bloco gerado pode alterar latência, portas ou comportamento.

B.3 Geração HDL e build para DE10-Nano

Após a programação do FPGA, o feed e o receptor podem ser executados na placa:

O endereço `0xFF200000` corresponde à base típica da ponte leve HPS–FPGA na DE10-Nano quando o componente está no offset zero do Platform Designer. Se o componente for posicionado em outro offset, a base deve ser ajustada.

Listagem 6 – Fluxo previsto para DE10-Nano

```
make de10-sysroot
make matlab-login
make build
make quartus-program
make de10-enable-bridges
make deploy
```

Fonte: Autoria própria (2026).

Listagem 7 – Execução na placa

```
ssh root@192.168.7.1 'cd /home/root && ./fast_data_feed'
ssh root@192.168.7.1 \
  'cd /home/root && HFT_FPGA_MMIO_BASE=0xFF200000 ./fast_receiver'
```

Fonte: Autoria própria (2026).

Tabela 7 – Variáveis de ambiente do receptor

Variável	Função
`HFT_FPGA_MMIO_BASE`	Base física do periférico. Quando ausente, o receptor opera em modo software-only.
`HFT_FPGA_MMIO_SPAN`	Tamanho da região mapeada. Quando ausente, usa valor padrão suficiente para a geometria esperada.
`HFT_FPGA_MMIO_DEV`	Dispositivo usado para mapeamento MMIO. Por padrão, `/dev/mem`.

Fonte: Autoria própria (2026).

B.4 Variáveis de ambiente

A presença de `HFT_FPGA_MMIO_BASE` muda o modo de operação do receptor. Por isso, testes de software devem ser executados sem essa variável, enquanto testes em placa devem defini-la explicitamente. Em ambientes compartilhados, recomenda-se imprimir a base efetiva antes de enviar eventos para evitar confusão entre offset de Platform Designer e base da ponte leve HPS–FPGA.

B.5 Sequência recomendada de depuração

Uma sequência prática de depuração é:

1. executar `make cpp-test` para validar a semântica de filas no wrapper;
2. executar `make cpp-smoke` para validar FAST, TCP e decodificação sem FPGA;
3. executar `make vhdl-test-all` para validar os blocos de hardware em simulação;
4. gerar e programar o bitstream;

5. habilitar pontes HPS–FPGA;
6. executar o receptor com `HFT_FPGA_MMIO_BASE` e verificar leitura de `MAGIC`;
7. iniciar o feed e observar respostas `[FPGA->ARM]`.

Se a leitura de `MAGIC` falhar, o problema provavelmente está em endereço, bitstream ou ponte HPS–FPGA. Se `MAGIC` estiver correto, mas `Send` falhar por fila cheia, deve-se investigar consumo TX no FPGA, reset e handshakes internos. Se respostas não aparecem, mas TX é consumida, o foco passa a ser livro de ofertas, estratégia e fila RX.